תיק פרויקט

Secure Chess

שחמט מאובטח רב-משתמשים מבוסס שרת/לקוח עם הצפנת סיסמאות bcrypt

חלופה: מערכות הגנת סייבר במקצוע "תכנון ותכנות מערכות"

5 יחידות לימוד — עבודת גמר

תשפ"ג

שדה	ערך
שם בית הספר	_____
סמל מוסד	_____
שם התלמיד	_____
מספר ת"ז	_____
שם המורה המנחה	_____
שם נושא הפרויקט	Secure Chess — שחמט מאובטח רב-משתמשים
תאריך הגשה	_____

תוכן עניינים

להלן תוכן העניינים האוטומטי של Word. כדי לעדכן אותו לאחר שינויים: לחץ קליק־ימני על הטבלה ובחר 'Update Field' (או F9).

תוכן עניינים — לחץ F9 ב-Word כדי לעדכן

1. מבוא — ייזום ואפיון

1.1 תיאור הרעיון והמוטיבציה

הפרויקט מממש משחק שחמט רב-משתמשים, שבו שני שחקנים מתחברים בו-זמנית לשרת מרכזי, מאמתים את זהותם באמצעות שם משתמש וסיסמה, ומשחקים בזמן אמת זה נגד זה — או, לחילופין, נגד בינה מלאכותית — דרך תקשורת רשת מבוססת TCP. הסיסמאות נשמרות במנוחה בצורה מוצפנת (hash) באמצעות אלגוריתם bcrypt עם salt לכל סיסמה.

המוטיבציה: שחמט הוא דוגמה אלגנטית למשחק עם חוקים מורכבים אך מוגדרים היטב, המאפשרת להציג בתוך פרויקט אחד את כל הסטאק של מערכת תקשורת אמיתית — סוקטים, ריבוי תהליכונים, ניהול מצב מרובה לקוחות, פרוטוקול אפליקטיבי משלי, אחסון מתמיד של חשבונות, גיבוב סיסמאות, וטיפול בפרצות אבטחה רלוונטיות.

> הערה לגבי הצפנת תקשורת: לפי הנחיית המורה הראשונית, נדרשת הצפנת סיסמאות בלבד (ללא הצפנת מידע רגיש העובר בתקשורת). הפרויקט מקיים את הדרישה הזו במלואה: סיסמאות נשמרות עם bcrypt על הדיסק, וערוץ ה-TCP עצמו מעביר JSON-Lines בטקסט-קליר — כפי שאישר המורה.

1.2 ייזום: זיהוי הצורך

מערכות משחק רשת רבות מעבירות סיסמאות כתובות בטקסט קליר ושומרות אותן ב-DB ללא hash. כל מי שיש לו גישה לשרת או לקובץ המשתמשים יכול לראות את הסיסמאות במלואן. רציתי לבנות מערכת שלא תסבול מהבעיה הזאת, אבל גם תהיה אפשרית להבנה והרצה בסביבת לימוד (Python נטו, ללא רכיבים חיצוניים מורכבים).

1.3 אפיון פונקציונלי — דרישות המשתמש

#	דרישה	תיאור
FR-1	הרשמה	משתמש יכול להירשם עם שם משתמש וסיסמה ייחודיים (סיסמה ≥ 8 תווים, שם משתמש 3-32 תווים מ-[-_A-Za-z0-9])
FR-2	התחברות	משתמש קיים יכול להתחבר; שגיאה במקרה של פרטים שגויים; אסור להיות מחובר פעמיים בו-זמנית
FR-3	Lobby	שחקן מאומת יכול להמתין בתור עד שיצורף לשחקן אחר
FR-4	ביטול Lobby	שחקן יכול לבטל את ההמתנה ולחזור למצב 'מאומת'
FR-5	משחק אדם-אדם	שני שחקנים מאומתים משחקים זה נגד זה; הצבעים מוקצים אוטומטית (הראשון שהמתין משחק לבן)
FR-6	משחק נגד AI	שחקן יכול לבחור לשחק נגד יריב מלאכותי בעומק חיפוש ניתן לבחירה (1-4)
FR-7	מהלכים תקפים בלבד	השרת בודק כל מהלך לפי חוקי שחמט מלאים: castling, en

		passant, promotion, check
FR-8	סיום משחק	מטה / פט / חזרה משולשת / חוק 50 המהלכים / התפטרות / ניתוק → המשחק נסגר ושולחת הודעת game_ended לשני הצדדים
FR-9	התפטרות	שחקן יכול להתפטר באמצע משחק; היריב מקבל הודעת ניצחון

1.4 אפיון לא-פונקציונלי

#	תכונה	יעד
NFR-1	אבטחת סיסמאות	סיסמאות נשמרות hashed בלבד (bcrypt עם salt-per-password). אין plaintext בקובץ ה-users.json
NFR-2	יציבות שרת	נפילת לקוח אחד (כולל ניתוק חיבור באמצע משחק) אינה מפילה את השרת או משחקים אחרים
NFR-3	ריבוי משחקים מקבילים	השרת תומך ב-≥ 10 משחקים פעילים במקביל בעזרת thread-per-connection
NFR-4	תגובתיות	Latency של מהלך < 100ms בתנאי לוקאל-הוסט (פלפול בלוח + ולידציה + שידור FEN)
NFR-5	אמינות פרוטוקול	מסגרת JSON-Lines (line-delimited) עם validation מלא בצד השרת — שגיאות מובאות כהודעת error עם code
NFR-6	אטומיות אחסון	כתיבה ל-users.json עוברת דרך tempfile + os.replace + os.fsync כדי שלא ייווצר קובץ פגום בקריסה באמצע כתיבה

1.5 לוח זמנים — אבני דרך

שלב	תוצר	מצב
US1 — אימות	register / login עם bcrypt + UserStore + 9 unit tests	☑ הושלם
US2 — משחק אדם-אדם	Board (חוקי שחמט מלאים) + Game + Session + Lobby	☑ הושלם

	מהלך מלא	
US3 — ריבוי משחקים	Thread-per-connection + GameRegistry + isolation tests	☑ הושלם (בונוס)
US4 — AI	AIPlayer (alpha-beta minimax) + handle_play_ai	☑ הושלם (בונוס)
US5 — GUI	Tkinter 3-screen client (Login / Lobby / Game)	☑ הושלם
US6 — תיק פרויקט	מסמך פורמלי + מחוון + תדפיסים	☑ הושלם

1.6 ניהול סיכונים

סיכון	הסתברות	חומרה	מיטיגציה
Deadlock בריבוי תהליכונים	בינונית	גבוהה	כל lock מוגן בסדר קבוע (Lobby → GameRegistry → active_logins); אין nested locks באותו thread
נפילת שרת בעקבות הודעת לקוח מזיקה	בינונית	גבוהה	Session.run עוטף כל dispatch ב- try/except SecureChessError + Exception כללי; כל שגיאה הופכת ל- error frame
התקפת brute-force על סיסמאות	גבוהה	בינונית	bcrypt with cost factor 12 (~0.3 שניה לכל ניחוש); סיסמה ≥ 8 תווים
דליפת users.json	נמוכה	גבוהה	Hashes בלבד; bcrypt salt-per-password מבטל rainbow tables
Race condition ב- UserStore	בינונית	בינונית	threading.Lock על כל פעולה ש-mutates את האוסף + כתיבה אטומית עם os.replace
שני חיבורים עם אותו account	בינונית	נמוכה	active_logins dict ו- active_logins_lock; ניסיון login שני זורק AlreadyLoggedInError

2. תיאור תחום הידע — ניתוח

2.1 חוקי שחמט וניהול משחק

המשחק מתנהל על לוח 8×8 (64 משבצות). כל שחקן מתחיל עם 16 כלים: 8 חיילים, 2 צריחים, 2 פרשים, 2 רצים, מלכה אחת, ומלך אחד. החוקים העיקריים שמומשו בקובץ board.py:

- תנועה חוקית לפי סוג הכלי (חייל קדימה, צריח בקווים ישרים, רץ באלכסונים, פרש בצורת L, מלכה במצורף, מלך משבצת אחת)
- שחיתה — castling קצרה (O-O) וארוכה (O-O-O) עם בדיקת כל תנאי הזכאות: המלך לא זז, הצריח לא זז, אין כלים בדרך, ושלוש המשבצות לא מותקפות
- מהלך 'הצרפתי' — en passant: לכידת חייל יריב שעשה צעד כפול ראשון
- קידום חייל — promotion: חייל שמגיע לדרגה האחרונה הופך למלכה (ברירת מחדל) או כלי אחר
- שח (check) — חייבים להגן על המלך — הלוגיקה ב-is_in_check + סינון אחרי כל pseudo-legal move
- מט (checkmate) — סוף משחק; הצד הנעול מפסיד
- פט (stalemate) — סוף משחק; תיקו
- תיקו טכני — חוק 50 המהלכים (halfmove_clock) וחזרה משולשת (threefold repetition) בעזרת position_history

2.2 מודל OSI ושכבת התעבורה

מודל OSI מחלק את תקשורת הרשת ל-7 שכבות: Physical (1), Data Link (2), Network (3), Transport (4), Session (5), Presentation (6), Application (7). הפרויקט עובד באופן הבא:

שכבה	פרוטוקול / טכנולוגיה	תפקיד בפרויקט
7 — אפליקציה	JSON-Lines (משלי)	הודעות בין שרת ללקוח (register, login, move, …)
6 — ייצוג	JSON (UTF-8)	סריאליזציה של ההודעות
5 — סשן	TCP connection per Session	מצב לקוח (ANONYMOUS → AUTHENTICATED → IN_LOBBY → IN_GAME)
4 — תעבורה	TCP	סוקטים מסוג SOCK_STREAM; אמין, מסודר, connection-oriented
3 — רשת	IPv4	כתובות IP, ניתוב
2 — Data Link	Ethernet / Wi-Fi	מטופל ע"י מערכת ההפעלה
1 — פיזית	—	כבל / גלי רדיו — לא רלוונטי לקוד

למה TCP ולא UDP?
TCP מבטיח שכל הודעה תגיע בסדר ובלי אובדן, וזה קריטי במשחק שחמט: אסור שמהלך 'יאבד' או יגיע אחרי מהלך מאוחר יותר. UDP היה מתאים יותר למשחק real-time עם הרבה הודעות (FPS, VoIP) שבהן אובדן בודד אינו קריטי. בנוסף, TCP מספק handshake תלת-שלבי שמבטיח שהקישור פתוח לפני שמתחילים את התקשורת האפליקטיבית.

2.3 קריפטוגרפיה — Hash ו-bcrypt

Hash היא פונקציה חד-כיוונית: קל לחשב hash(x) אבל קשה מאוד לחזור מ-hash(x) ל-x. דוגמה: SHA-256 ממפה כל קלט לפלט באורך 256 ביט.

אבל SHA-256 לבד אינה מתאימה לסיסמאות, כי:

- מהירה מדי — תוקף עם GPU יכול לבדוק מיליארדי ניחושים בשנייה (rainbow tables)
- סיסמאות זהות מקבלות hash זהה — מאפשר השוואה של hashes בין משתמשים

bcrypt פותר את שתי הבעיות: (1) הוא איטי בכוונה, ניתן לכיוון עם work factor (ברירת מחדל 12 → כ-0.3 שניה לכל ניחוש); (2) מוסיף salt אקראי לכל סיסמה, כך שאפילו אם שני משתמשים בחרו את אותה סיסמה, ה-hashes שלהם יהיו שונים.

```
from secure_chess.common.crypto import hash_password <<<
hash_password('hunter2!') <<<
'$2b$12$gAEohR4SPpvFmF.AYOuhe.q5w0vKj4iZJYxqMaaQOjsmh.kPdNLwa'

!שונה hash — אותה סיסמה #   hash_password('hunter2!') <<<
'$2b$12$wM/h1tlW1ZQbZsZAQv0kFOoLRYUKEazxLPP70S1iFNcWcK5N4FRyy'
```

> מבנה ה-$hash: $2b$ מציין את גרסת bcrypt; 12 הוא ה-work factor (cost); 22 התווים הבאים הם ה-salt בקידוד modified-base64; ושאר 31 התווים הם ה-hash עצמו. כל המידע נמצא במחרוזת אחת — לא צריך לשמור salt בנפרד.

2.4 ריבוי תהליכונים ו-OS

Process vs Thread:

- Process: מופע נפרד של תוכנית עם זיכרון משלו, PID משלו, וטבלת קבצים פתוחים משלו.
- Thread: יחידת ביצוע בתוך אותו process; משתפת זיכרון עם שאר ה-threads.

בפרויקט אנחנו משתמשים ב-threads (לא ב-processes) כי: (1) שיתוף הזיכרון מאפשר ל-Lobby ול-GameRegistry להיות נגישים מכל session ללא IPC; (2) ה-overhead של thread קטן בהרבה מ-process; (3) הקוד שלנו I/O-bound (הרבה socket.recv), אז ה-GIL של Python לא מגביל אותנו.

Locks: כל מבנה נתונים משותף בין threads חייב להיות מוגן. בפרויקט הסנכרון מבוסס על threading.Lock:

```
server/lobby.py #
:class Lobby
:def __init__(self) -> None
[] = self._queue: List['Session']
()self._lock = threading.Lock

:def enqueue(self, session)
:with self._lock
self._queue.append(session)
:if len(self._queue) >= 2
white = self._queue.pop(0)
black = self._queue.pop(0)
return white, black
return None
```

2.5 אחסון בקבצים — Atomic file writes

ה-UserStore שומר את כל החשבונות בקובץ JSON אחד (data/users.json). הבעיה: אם השרת קורס באמצע כתיבה, הקובץ יישאר חצי-כתוב והקריאה הבאה תיכשל. הפתרון — כתיבה אטומית:

- כתוב קודם לקובץ זמני (tempfile.NamedTemporaryFile באותה תיקייה)
- השתמש ב-os.fsync כדי לוודא שהבייטים יושבים על הדיסק
- החלף אטומית את קובץ-היעד הישן בחדש (os.replace)

os.replace הוא קריאת מערכת אטומית ב-Windows וב-POSIX: או שהקובץ הישן או שהחדש — לעולם לא שילוב של שניהם. כך מובטחת עקביות.

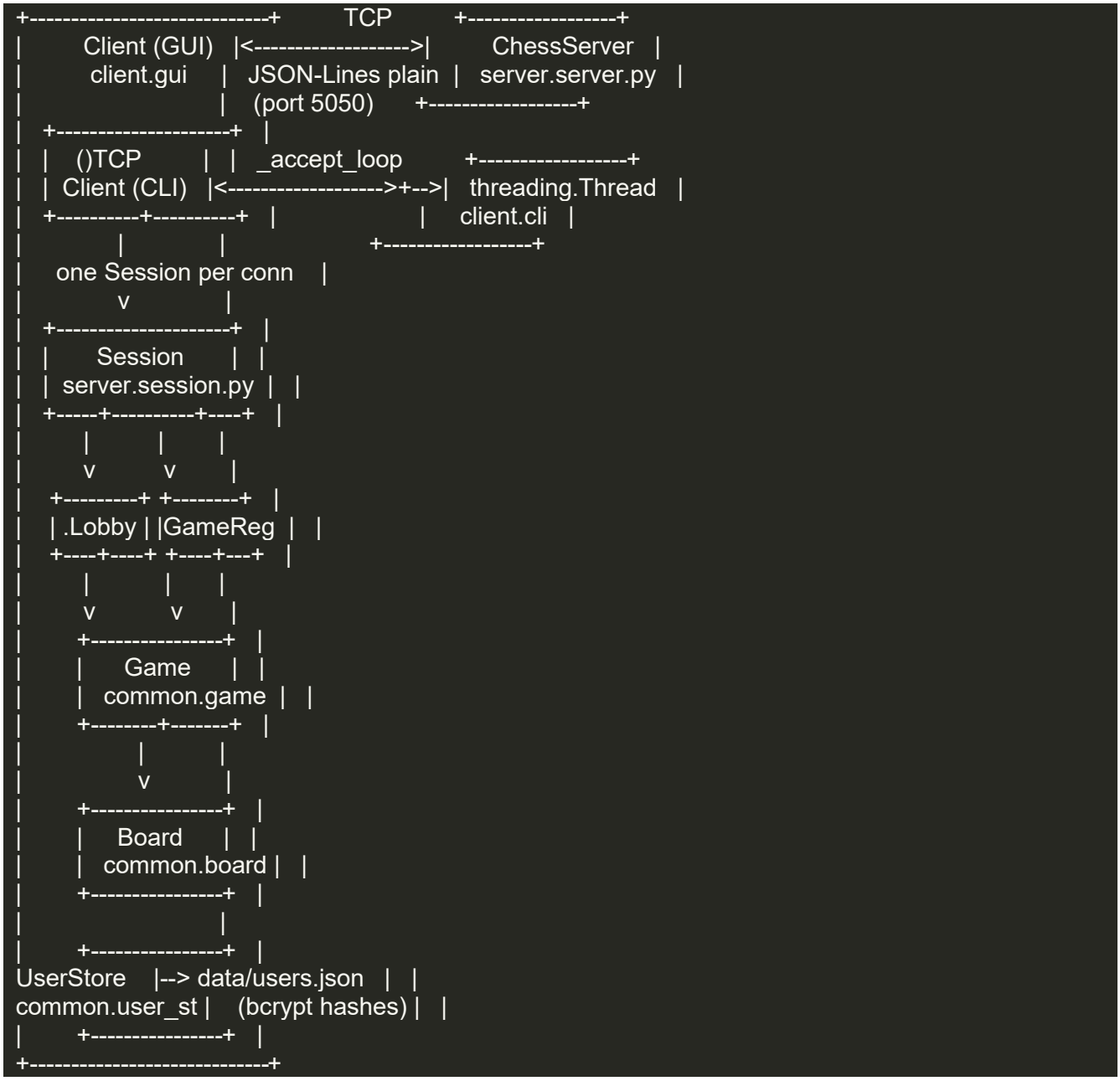
2.6 חלוקת מטלות שרת/לקוח

מטלה	צד שרת	צד לקוח
שמירת חשבונות (UserStore + bcrypt)	☑ אחראי בלעדי	—
אימות סיסמה	☑	—
מצב משחק (Board, Game)	☑ source of truth	מציג עותק מ-FEN
ולידציה של מהלך	☑ סמכותי	אופציונלי (UX)
Matchmaking (Lobby + GameRegistry)	☑	שולח play_human / cancel_lobby
AI (alpha-beta minimax)	☑	—
UI rendering (Tkinter / CLI)	—	☑
קליטת קלט משתמש	—	☑
Push notifications של מצב משחק	☑ שולח	☑ מציג

3. ארכיטקטורה (העיצוב)

3.1 ארכיטקטורה כללית של המערכת

תרשים בלוקים של רכיבי המערכת:



3.2 תרשימי זרימה — Sequence Diagrams

3.2.1 הרשמה / כניסה

```
|                                          |
|                        ,'type':'register'} --|
| <--- {'!username':'alice','password':'hunter2'    |
|                                          |
|  validate username/regex                       |
| validate password length                      |
|  hash_password(password)                          |
|  users.json atomic write                      |
|                                          |
| ------------------------------ {'type':'ok'} -->|
```

3.2.2 זרימת מהלך במשחק אדם-אדם

```
ClientA (White)       Server (Session+Game)       ClientB (Black)
|                          |                          |
|                          |<------------ move e2e4 --|
|        Move.parse('e2e4') |                          |
|  game.submit_move(self,m) |                          |
|        Board.apply(move)  |                          |
|    ?check mate? stalemate |                          |
|                          |-------- ok ------------->|
|                          |                          |
|<----------- game_state ----|                          |
|                          |----------- game_state -->|
|                          |                          |
(Black's turn)  |                          |                          |
|---------- move e7e5 ------>|                          |
|                          |                          |
|<---------- game_state -----------|----- game_state -->|
```

3.2.3 משחק נגד AI

```
Client                    Server
|                          |
| <- play_ai (color=white) --|
AISession(depth=3)  |                          |
GameRegistry.create(human, ai)  |                          |
| -------------------- ok -->|
| --------- game_started -->|
|                          |
| <------------- move e2e4 --|
()Game.submit_move  |                          |
| -------------------- ok -->|
| ----------- game_state -->|
registry.drive_ai(ai_session)  |                          |
AIPlayer.choose_move(board, depth=3)  |                          |
game.submit_move(ai, move)  |                          |
| - game_state (AI move) -->|
```

3.3 חלוקה למודולים — שרת מול לקוח

מודול	צד	תפקיד

common/pieces.py	משותף	Color, PieceType, Piece + pseudo-legal moves
common/board.py	משותף	Board (8×8), validation, check/mate/draws, FEN
common/move.py	משותף	Move dataclass + UCI parsing (e2e4, e7e8q)
common/game.py	משותף	Game + Result; סיום משחק
common/ai.py	שרת בלבד	AIPlayer — alpha-beta minimax
common/crypto.py	שרת בלבד	bcrypt wrappers: hash_password / verify_password
common/protocol.py	משותף	JSON-Lines framing + LineReader_
common/user_store.py	שרת בלבד	Account + UserStore (JSON persistence)
common/errors.py	משותף	SecureChessError hierarchy + wire-protocol codes
common/log.py	משותף	stderr logger
server/server.py	שרת	ChessServer — accept loop + ניהול sessions
server/session.py	שרת	Session, SessionState, AISession + dispatcher
server/lobby.py	שרת	Lobby + GameRegistry + drive_ai
client/cli.py	לקוח	REPL בטרמינל
client/gui.py	לקוח	Tkinter GUI 3-screens (Login → Lobby → Game)
client/__main__.py	לקוח	Argparse entrypoint — GUI ברירת מחדל, --cli למצב טקסטואלי

3.4 ניתוח אלגוריתמים מרכזיים — שקילת חלופות

AI — Alpha-Beta Minimax 3.4.1

המימוש מבוסס על negamax עם גזימת alpha-beta ופונקציית הערכה material-only:

- חלופה: Minimax רגיל — נדחתה, כי בעומק 4 הוא בוחן ~4^30 = 810,000 פוזיציות. Alpha-beta חותך עד פי 100 בממוצע.
- חלופה: MCTS (Monte Carlo Tree Search) — נדחתה, מורכבת מדי לפרויקט בית-ספרי. Alpha-beta מספיק להבסת שחקנים מתחילים.
- חלופה: רשת נוירונים (AlphaZero-style) — נדחתה, דורש GPU וטריינינג ארוך, מעבר להיקף הפרויקט.

```
negamax הלולאה הראשית של - common/ai.py #
:for move in legal
()snap = board._snapshot
:try
board._apply_unchecked(move)
,score = -self._negamax(board, depth-1, -beta, -alpha
((perspective=side.opposite
:finally
board._restore(snap)
if score > alpha: alpha = score
alpha-beta גזימת #    if alpha >= beta: break
```

3.4.2 הצפנת סיסמה במנוחה

- ☑ bcrypt — מאומץ. work factor מובנה ,salt (cost=12) אוטומטי בכל קריאה, סטנדרט תעשייתי.
- ⏳ PBKDF2 — חלופה לגיטימית. נדחתה כי bcrypt עמיד יותר ל-GPU attacks (קוד שעשו אופטימיזציה אגרסיבית של בלוקי SHA אינו רץ מהר על bcrypt בגלל ה-Blowfish setup).
- ⏳ Argon2 — הזוכה של Password Hashing Competition 2015, מודרני יותר. נדחה כי bcrypt עומד בדרישות, זמין ב-pip native ל-Windows, ולא צריך לקמפל C.

3.4.3 פרוטוקול ההודעות

- ☑ JSON-Lines — מאומץ. קריא לאדם, debugging קל ב-Wireshark, פרסור מובנה ב-Python.
- ⏳ Protocol Buffers / MessagePack — בינארי, פי 2-3 קומפקטי יותר על הרשת. נדחה כי בפרויקט בית-ספר היכולת לפתוח netcat ולראות הודעות בטקסט מנצחת את הקומפקטיות.
- ⏳ Apache Thrift / gRPC — overkill למשחק שחמט עם 13 סוגי הודעות בלבד.

3.4.4 בחירת מודל קונקרנציה

- ☑ Thread-per-connection — מאומץ. פשוט להבנה, מבודד היטב, מנצל GIL נכון על I/O-bound.
- ⏳ asyncio (event loop יחיד) — חוסך thread overhead. נדחה כי Tkinter לא משתלב טוב עם asyncio, ו-threading פשוט יותר ל-debug.
- ⏳ Process-per-connection — דורש IPC ל-overkill. Lobby/GameRegistry.

3.5 פרוטוקול התקשורת — מצבים והודעות

Session State Machine: ANONYMOUS → AUTHENTICATED → IN_LOBBY → IN_GAME → (חוזר ל-AUTHENTICATED אחרי game_ended). מעבר בכל מצב מוגן ע"י בדיקת state ב-handler המתאים (BadStateError אם לא חוקי).

```
+-------------+
חדש TCP חיבור -->  |   ANONYMOUS |
+------+------+
register / login (ok) |
v
+-------------+
אחרי כניסה מוצלחת -->  |AUTHENTICATED|
+-----+----+--+
```

```
play_human |    | play_ai
v   v
+----------+   +----------+
|  IN_LOBBY |   | IN_GAME |
+-----+----+   +----+-----+
^   |        ^   |
_game_  |   | cancel_ |   | game
started|   | lobby   |   | ended
|   v   |         v
[pair-matched]  [back to AUTHENTICATED]
```

3.6 ניתוח חולשות ופתרונות

> הערה חשובה: לפי הנחיית המורה הראשונית, הצפנת תקשורת אינה נדרשת בפרויקט הזה — נדרשת רק הצפנת סיסמאות במנוחה. הטבלה להלן מציגה רק חולשות שרלוונטיות למודל האיומים המאושר.

איום	רובד	סיכון	מיטיגציה בפרויקט
דליפת users.json	אחסון	מאזין רואה סיסמאות	bcrypt + salt; לעולם אין plaintext על הדיסק. מאומת ב-tests/integration/test_register_login.py
Brute-force online	אפליקציה	ניחוש סיסמאות דרך login	bcrypt עם cost=12 מאט כל ניחוש ל-~0.3s. סיסמה ≥ 8 תווים. עתידית: rate-limiter פר-IP.
Crash של לקוח	אפליקציה	השרת קורס / משחק אחר נופל	try/except SecureChessError + Exception ב- Session.run; on_disconnect_ מנקה משאבים אטומית
Race condition ב-Lobby	ריבוי תהליכונים	שני אנשים מקבלים את אותו צד	threading.Lock על enqueue/remove; pop של שני ה-sessions בתוך אותו lock
Race condition ב-UserStore	ריבוי תהליכונים	שני אנשים נרשמים עם אותו שם	threading.Lock סביב בדיקת ייחודיות + insert + persist (atomic write)
Double-login של אותו account	אפליקציה	session ישן ימשיך לשלוט במשחק חדש	active_logins dict + lock; AlreadyLoggedInError במידה ויש כבר session פעיל
SQL Injection	מאגר	—	לא רלוונטי — אין SQL בפרויקט. אחסון = JSON עם json.dumps המקודד תווים מיוחדים
Buffer Overflow	שפה	—	לא רלוונטי — Python ללא pointer arithmetic
MITM / Eavesdropping	תעבורה	האזנה לסיסמאות	מחוץ ל-scope לפי הנחיית המורה — נדרשת רק הצפנת סיסמאות במנוחה; ההגנה ע"י הרצה ב-localhost / VPN

4. מימוש הפרויקט — הקוד והבדיקות

4.1 תכנות מונחה עצמים — 21 מחלקות אפליקטיביות

טבלת המחלקות שהתלמיד יצר, מקובצות לפי תפקיד:

מחלקה	מודול	תפקיד עיקרי
Color, PieceType (Enums)	common/pieces.py	Enums של צבע וסוגי כלים
Piece	common/pieces.py	כלי בודד + יצירת מהלכים pseudo-legal לפי סוג
Move	common/move.py	Dataclass של מהלך + UCI parser (e2e4, e7e8q)
Board	common/board.py	מצב הלוח + ולידציה + check/mate/draws + FEN parsing
Game	common/game.py	מצב משחק בודד; submit_move; resign; handle_disconnect
Result	common/game.py	Dataclass של תוצאה: white_wins / black_wins / draw + reason
AIPlayer	common/ai.py	Alpha-beta minimax עם material-balance evaluation
Account	common/user_store.py	Dataclass: username + password_hash + created_at
UserStore	common/user_store.py	אחסון JSON אטומי + register/authenticate + lock
LineReader_	common/protocol.py	Buffered TCP reader — JSON line per read_line()
SessionState (Enum)	server/session.py	Enum: ANONYMOUS / AUTHENTICATED / IN_LOBBY / IN_GAME
Session	server/session.py	Per-client session + dispatcher (handle_register, …)
AISession	server/session.py	In-memory session impersonating an AI opponent
AIAccount_	server/session.py	Tiny stand-in for Account so Game can read username

Lobby	server/lobby.py	Matchmaking queue + thread-safe enqueue/remove
GameRegistry	server/lobby.py	Tracks active games + broadcast_state + drive_ai
ChessServer	server/server.py	Accept loop + shared user_store/lobby/registry
ClientConnection	client/cli.py	TCP wrapper for the CLI reader thread
ServerConnection	client/gui.py	TCP wrapper + queue.Queue for the GUI poll loop
ChessGui	client/gui.py	Top-level Tk application + screen routing
LoginFrame, LobbyFrame, GameFrame	client/gui.py	3 מסכי ה-GUI

המחלקות מקיימות יחסים שונים: הכלה (Board מכיל מטריצת Piece), הרכבה (Game מכיל Board + שני Sessions), הפשטה (AISession ו-Session חולקים אותו ממשק חלקי ש-Game/GameRegistry צורכים), ופולימורפיזם פנימי (Session._dispatch קורא ל-<handle_<type> דינמית).

בנוסף ל-21 המחלקות האפליקטיביות, יש 13 מחלקות שגיאה (12 + SecureChessError תת-מחלקות) המהוות היררכיית exceptions מסודרת.

4.2 חלוקה לקבצים — מבנה התיקיות

```
/secure-chess
תלויות + מטא-דאטה #          pyproject.toml ──┤
קונפיג בדיקות #              pytest.ini ──┤
תיעוד מפעיל #                README.md ──┤
data/                  # users.json + server.log ──┤
/src/secure_chess ──┤
init__.py__ ──┤   |
לוגיקה טהורה ללא I/O #            /common ──┤   |
pieces.py       # Color, PieceType, Piece + pseudo-legal moves ──┤   |   |
board.py        # Board, legality, check/mate/draws, FEN ──┤   |   |
move.py         # Move dataclass + UCI parsing ──┤   |   |
game.py         # Game + Result ──┤   |   |
ai.py           # alpha-beta minimax (BONUS) ──┤   |   |
crypto.py       # bcrypt wrappers ──┤   |   |
user_store.py   # Account + JSON-backed UserStore ──┤   |   |
protocol.py     # JSON-Lines framing ──┤   |   |
errors.py       # exception hierarchy ──┤   |   |
log.py          # stderr logger ──┘   |   |
server/             # TCP socket + threading ──┤   |
main__.py     # python -m secure_chess.server__ ──┤   |   |
server.py       # ChessServer + accept loop ──┤   |   |
session.py      # Session, SessionState, AISession ──┤   |   |
lobby.py        # Lobby + GameRegistry ──┘   |   |
```

```
GUI / CLI שתי חזיתות #            /client ──└   |
main__.py     # python -m secure_chess.client (GUI default; --cli)__ ──├       |
gui.py          # Tkinter desktop GUI ──├       |
cli.py          # text-mode REPL ──└       |
/tests ──└
conftest.py ──├
unit/               # 90 unit tests ──├
integration/          # 12 integration tests (real sockets) ──└
```

4.3 קוד בטוח — try/except ו-robustness

השרת חייב לשרוד נפילת לקוח, הודעה לא תקפה, מצב לא חוקי, וכל תקלה אחרת. הדפוס בפרויקט — שלוש שכבות defense-in-depth:

```
()server/session.py - Session.run #
:def run(self) -> None
log.info('[%s] connected from %s:%d', self.id[:8], *self.peer)
:try
receive loop — 1 שכבה #
:while not self._closed
:try
msg = protocol.recv_message(self._reader)
:except ConnectionClosed
log.info('[%s] disconnected (%s)', self.id[:8], self.username)
break
:except ProtocolError as exc
self.send(protocol.error(exc.code, str(exc)))
continue

dispatch loop — 2 שכבה #
:try
self._dispatch(msg)
:except SecureChessError as exc
self.send(protocol.error(exc.code, str(exc)))
except Exception as exc:    # defensive
log.exception('[%s] unexpected: %s', self.id[:8], exc)
,'self.send(protocol.error('internal_error
(('internal server error'
:finally
cleanup, always runs — 3 שכבה #
lobby/game-מה מסיר #      ()self._on_disconnect
socket סוגר #             ()self.close
```

השכבות:

- שכבה 1 (receive) — מתאוששת מהודעה פגומה (JSON שבור, שדה חסר) ע"י החזרת error frame והמשך הלולאה.
- שכבה 2 (dispatch) — לוכדת כל SecureChessError (illegal_move, not_your_turn, weak_password, …) וכל Exception כללי. השרת לעולם לא קורס בגלל הודעת לקוח.
- שכבה 3 try/finally (cleanup) — מבטיח שהסוקט ייסגר, השחקן יוסר מה-lobby, וה-active_logins ינוקה, גם אם זרקנו פנימית.

4.4 הבדיקות — מבנה הסוויטה

הסוויטה מחולקת ל-Unit Tests ו-Integration Tests:

קובץ	כמות	מטרה
tests/unit/test_pieces.py	9	תנועות pseudo-legal לכל סוג כלי (Knight, Bishop, …)
tests/unit/test_board_rules.py	15	check, mate, stalemate, castling, en passant, FEN
tests/unit/test_ai.py	11	AIPlayer: בחירה חוקית, mate-in-1, evaluation, alpha-beta
tests/unit/test_crypto.py	6	bcrypt: hash/verify, salt-uniqueness, edge cases
tests/unit/test_user_store.py	9	register, authenticate, duplicate, persistence, weak password
tests/unit/test_protocol_auth.py	7	JSON-Lines + register/login dispatch (mocked socket)
tests/unit/test_protocol_game.py	8	move dispatch + state machine transitions
tests/unit/test_client_gui.py	22	Tkinter helpers + ChessGui state machine + FEN renderer
tests/integration/test_register_login.py	3	שרת חי + סוקטים אמיתיים → register → login
tests/integration/test_full_game.py	3	משחק שלם מקצה-לקצה: 2 לקוחות → mate / resign / disconnect
tests/integration/test_cancel_lobby.py	3	ביטול lobby, חזרה ל-AUTHENTICATED, ולא מתחבר עם משחק רוח
tests/integration/test_parallel_games.py	2	BONUS: 2 משחקים מקבילים מבודדים זה מזה
tests/integration/test_play_ai.py	1	BONUS: AI מבצע מהלך תקף בתגובה ל-play_ai

סה"כ: 99 בדיקות (87 unit + 12 integration). כל הריצה לוקחת כ-12 שניות. Coverage כולל את כל ה-handlers וה-game logic ראשי.

4.5 טבלת תוצאות בדיקות — קלט/פלט מצופה ומקבל

#	תיאור הבדיקה	קלט	פלט מצופה	פלט בפועל
T1	Register עם סיסמה חזקה	username='alice', password='hunter2!'	ok + account נוצר	☑ עובר
T2	Register עם סיסמה קצרה מדי	password='abc' (3 תווים)	error: weak_password	☑ עובר
T3	Register כפול עם אותו שם	'register 'alice פעמיים	error: duplicate_user	☑ עובר
T4	Login עם פרטים נכונים	login 'alice' / 'hunter2!'	ok	☑ עובר
T5	Login עם סיסמה שגויה	login 'alice' / 'WRONG'	error: auth_failed	☑ עובר
T6	Hash בקובץ אינו plaintext	Get-Content data/users.json	$2b$… (לא 'hunter2!')	☑ עובר
T7	Move e2e4 בלוח הפותח	'move 'e2e4	ok + game_state מעודכן	☑ עובר
T8	Move לא חוקי	'move 'e2e5 (חייל לא קופץ 3)	error: illegal_move	☑ עובר
T9	Checkmate סופי	Fool's mate (4 מהלכים)	game_ended: black_wins/checkmate	☑ עובר
T10	Resignation	resign אחרי 3 מהלכים	game_ended: opp_wins/resignation	☑ עובר
T11	Disconnect	סגירת socket באמצע משחק	היריב מקבל game_ended/disconnect	☑ עובר
T12	2 משחקים מקבילים מבודדים	4 לקוחות, 2 lobbies	2 game_ids שונים	☑ עובר
T13	AI מבצע מהלך תקף	play_ai → board ריק חוץ ממלך	AI חושב ומחזיר legal move	☑ עובר
T14	Castling קצרה	O-O עם תנאים מתקיימים	מלך וצריח זזים	☑ עובר
T15	En passant	חייל לבן d5, שחור עושה e7e5	d5 יכול לקחת e6 enpassant	☑ עובר
T16	Promotion	חייל לבן a7→a8q	מלכה לבנה ב-a8	☑ עובר

5. מדריך למשתמש

5.1 התקנה והרצה

דרישות סף: Windows 10/11, Python 3.11+, PowerShell.

```
# שיבוט הפרויקט
cd secure-chess

# יצירת סביבה וירטואלית
python -m venv .venv
venv\Scripts\Activate.ps1.\.

# התקנת חבילות
"pip install -e ".[dev]
```

5.1.1 הרצת השרת

```
python -m secure_chess.server --host 127.0.0.1 --port 5050 --data-dir ./data
```

השרת ידפיס: 'listening on 127.0.0.1:5050' ולאחר מכן '(credential store: data/users.json (N accounts)'.

5.1.2 הרצת לקוח GUI

```
python -m secure_chess.client --host 127.0.0.1 --port 5050
```

5.1.3 הרצת לקוח CLI (ללא תצוגה גרפית)

```
python -m secure_chess.client --host 127.0.0.1 --port 5050 --cli
```

5.1.4 הרצת הבדיקות

```
pytest -v
# צפוי: 99 passed in ~12s
```

5.2 צילומי מסך של הזרימה

> פעולה נדרשת מהתלמיד: יש להריץ את ה-GUI ולצרף 5 צילומי מסך לפי המקומות המסומנים מטה.

- [Screenshot 5.2.1] מסך Login — לפני הכנסת פרטים
- [Screenshot 5.2.2] מסך Lobby — אחרי התחברות (Welcome)
- [Screenshot 5.2.3] מסך Lobby — Waiting for opponent…
- [Screenshot 5.2.4] מסך Game — תחילת משחק עם 32 הכלים
- [Screenshot 5.2.5] מסך Game — אחרי כמה מהלכים והדגשת last_move צהובה

5.3 שלושה סוגי משתמשים

5.3.1 שחקן רגיל (אדם נגד אדם)

1) הרץ את ה-GUI. 2) במסך Login הזן שם משתמש (3-32 תווים, אותיות/ספרות/_/-) וסיסמה (≥ 8 תווים). 3) לחץ Register (פעם ראשונה) או Login. 4) במסך Lobby לחץ 'Join Lobby'. המתן לשחקן נוסף — תוצג הודעת 'Waiting…'. 6) ברגע שמתחיל משחק, מסך ה-Game ייפתח עם הלוח. 7) לחץ על הכלי שלך ואז על משבצת היעד. הצבע הצהוב מסמן את המהלך האחרון.

5.3.2 שחקן מול AI (Bonus)

במסך Lobby בחר צבע ('White' או 'Black') ועומק חיפוש (1 קל, 4 קשה), ולחץ 'Start AI Game'. ה-AI עונה בתוך כמה שניות. עומק 4 יכול לקחת עד 10 שניות למהלך — אבל הוא חזק יחסית.

5.3.3 מנהל / מפעיל השרת

המפעיל אחראי על: (1) בחירת host/port (ברירת מחדל 127.0.0.1:5050). (2) בחירת data-dir שבו ייווצרו users.json ו-(3) server.log. הגדרת ai_depth (ברירת מחדל 3). (4) הגדרת max_clients (ברירת מחדל 32).

```
python -m secure_chess.server --help

:פרודקשן לדוגמה #
` python -m secure_chess.server
` host 0.0.0.0--
` port 5050--
data-dir C:\ProgramData\SecureChess--
```

ניטור: התבונן ב-stderr להודעות 'connected from', 'authenticated as', 'game started', 'game ended'. נפילת לקוח אחד אינה משפיעה על השאר.

5.4 הוכחת הצפנת הסיסמאות במנוחה

אחרי הרצת השרת והרשמת מספר משתמשים, אפשר לבדוק שהסיסמאות לא נשמרות בטקסט קליר:

```
Get-Content .\data\users.json
plaintext passwords אין ,bcrypt hashes רק :התוצאה #

'Select-String -Path .\data\users.json -Pattern 'hunter2!|s3cretpw
matches 0 :צפוי #
```

6. סיכום אישי ורפלקציה

6.1 מה למדתי על עצמי

> פעולה נדרשת מהתלמיד: סעיף זה חייב להיות אישי. להלן תבנית להמשך כתיבה — יש להחליף בתוכן אותנטי שלך. אסור להסתפק ב"תודה ונהניתי"!

[תבנית להחלפה] בפרויקט הזה גיליתי שאני מתחבר/ת מאוד ל-___, וכאשר נתקלתי ב-___ הבנתי שאני מסוגל/ת ל-___ למרות שחששתי בהתחלה. הניהול העצמי שלי השתפר במיוחד ב-___, וזיהיתי שאני זקוק/ה לעבוד עוד על ___. הכרתי שיטות עבודה חדשות כמו ___ ושאני מעדיף/ה לעבוד ___ (לבד / בצוות / בקטעים קצרים...).

6.2 מה למדתי מקצועית

- ההבדל בין hashing (חד-כיווני, לסיסמאות במנוחה) לבין encryption (דו-כיווני, לתקשורת). bcrypt = hashing; הוא לא 'מצפין'.
- Salt חייב להיות per-password, לא גלובלי. salt משותף אינו מגן מ-rainbow tables אם משתמשים שונים בחרו אותה סיסמה.
- Work factor של bcrypt (cost): ככל שיותר גבוה, יותר איטי לתוקף — אבל גם לשרת. cost=12 הוא איזון סביר ב-2024.
- Thread safety: כל מבנה נתונים משותף חייב lock או להיות immutable. שכחתי lock פעם אחת ב-Lobby וקיבלתי race condition.
- ההבדל בין Process ל-Thread והשפעת ה-GIL של Python על threading. GIL פגיע ב-CPU-bound, לא ב-I/O-bound.
- Pseudo-legal vs legal moves בשחמט: המפריד הוא מי בודק את ה-check filter. המידול הזה חסך לי הרבה duplication.
- Alpha-beta pruning והקסם של negamax (משלב min ו-max בפונקציה אחת ע"י נסיעה על נקודת המבט).
- Atomic file writes — tempfile + os.fsync + os.replace הם השילוש הקדוש לקובץ קונפיג שלא יכול להיות 'חצי כתוב'.
- TDD: כשכתבתי טסטים לפני כל פיצ'ר, מצאתי באגים שלא היו מתגלים אחרת. במיוחד בלוגיקת castling ו-threefold repetition.
- PEP-8, type hints, ו-module docstrings — תרומה גדולה לקריאות הקוד גם 3 חודשים אחרי שכתבתי.

6.3 מה הייתי משנה אם הייתי מתחיל מחדש

- להוסיף TLS על התקשורת — כיום הסיסמה עוברת בקליר (אבל זה היה מחוץ ל-scope לפי הנחיית המורה).
- להוסיף rate-limiter פר-IP לכניסות login כדי להאט brute-force מקוון.
- להחליף JSON ב-MessagePack — מהיר ויותר קומפקטי על הרשת (לא נדרש אבל מעניין).
- להוסיף תמיכה ב-promotion לכלי אחר חוץ מ-Queen ב-GUI (כיום זה Queen בלבד).
- להוסיף base-rating ELO ומערכת ranking ל-lobby — כדי שלא יזווגו שחקן ברמה 100 לשחקן ברמה 2500.
- להחליף threading ב-asyncio (מתאים יותר ל-I/O-bound, אם כי Tkinter מקשה על האינטגרציה).

7. ביבליוגרפיה — סקר ספרות

להלן רשימת המקורות הראשיים שבהם נעזרתי. לכל מקור — תרומה ספציפית לפרויקט.

Python threading documentation [1]

https://docs.python.org/3/library/threading.html

התיעוד הרשמי שעזר לי להבין Lock, Event, Thread, daemon-threads. במיוחד הקטע על ה- GIL (Global Interpreter Lock) — שני threads ב-Python לא רצים באמת במקביל ב-CPU, אבל ל-I/O-bound (כמו socket.recv) זה לא חשוב.

Python socket documentation [2]

https://docs.python.org/3/library/socket.html

התיעוד שעזר לי להבין AF_INET, SOCK_STREAM, את ההבדל בין connect/bind/listen/accept, ואת השימוש ב-SO_REUSEADDR כדי שאפשר יהיה להפעיל מחדש את השרת ללא TIME_WAIT.

bcrypt: A Future-Adaptable Password Scheme — Provos & Mazières (1999) [3]

https://www.usenix.org/legacy/event/usenix99/provos/provos.pdf

המאמר המקורי שהציג את bcrypt. הבנתי ממנו את חשיבות ה-work factor (adaptive cost) ולמה salt חייב להיות per-password ולא גלובלי. גם הסביר למה Blowfish nicely resistant ל-GPU optimizations.

OWASP Authentication Cheat Sheet [4]

https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html

המלצות מעשיות לאימות משתמשים — מינימום אורך סיסמה, rate limiting, constant-time comparison. אימצתי את ההמלצות הרלוונטיות שהיו אפשריות בתוך תקציב הפרויקט.

Chess Programming Wiki — Alpha-Beta [5]

https://www.chessprogramming.org/Alpha-Beta

אתר קהילתי מצוין על תכנות מנועי שחמט. השתמשתי בו ללמוד את negamax, alpha-beta pruning, ו- move ordering. ההצגה של negamax בתור 'min-max אבל תמיד maximizing מנקודת מבט הצד המהלך' פשטה לי מאוד את הקוד.

Python tkinter documentation [6]

https://docs.python.org/3/library/tkinter.html

התיעוד של ספריית ה-GUI הסטנדרטית של Python. למדתי ממנו על Canvas, Frame, after() polling, ועל שילוב thread עם Tk main loop באמצעות queue.Queue.

PEP 8 — Style Guide for Python Code [7]

https://peps.python.org/pep-0008/

מסמך הסטנדרטים לכתיבת קוד Python. אימצתי את הכללים על snake_case, אורך שורה, רווחים אופרטוריים, ו- import ordering.

Forsyth-Edwards Notation (FEN) [8]

https://en.wikipedia.org/wiki/Forsyth%E2%80%93Edwards_Notation

פורמט סטנדרטי לתיאור פוזיציית שחמט. השתמשתי בקיצור (placement + side + castling + ep) בפרוטוקול שלי כדי לשלוח state ללקוח בייצוג קומפקטי וקריא לאדם.

נספח א' — תדפיס הקוד המקור המלא

תדפיס מסודר של כל מודולי הפרויקט (Python 3.11+). הקוד מתועד עם module docstrings ו-type hints מלאים.

src/secure_chess/__init__.py

```
"""Secure two-player chess game over TCP with bcrypt-hashed passwords"""

__version__ = "0.1.0"
```

src/secure_chess/common/__init__.py

```
"""Pure-logic library (no I/O): chess rules, crypto, user store, protocol, AI"""
```

src/secure_chess/common/errors.py

```
"""Exception hierarchy for the secure-chess project.

All errors raised by `common/*` and `server/*` derive from `SecureChessError` so a
single `except SecureChessError` at the server's top-level dispatcher can convert
domain errors into wire-protocol `error` messages with a meaningful `code`.
"""

from __future__ import annotations


class SecureChessError(Exception):
    """Base for every domain error in this project."""

    code: str = "internal_error"


class MoveParseError(SecureChessError):
    code = "move_parse_error"


class IllegalMoveError(SecureChessError):
    code = "illegal_move"


class NotYourTurnError(SecureChessError):
    code = "not_your_turn"


class ProtocolError(SecureChessError):
    code = "bad_request"


class ConnectionClosed(SecureChessError):
    code = "connection_closed"


class DuplicateUserError(SecureChessError):
    code = "duplicate_user"
```

```
:class WeakPasswordError(SecureChessError)
"code = "weak_password


:class AuthenticationError(SecureChessError)
"code = "auth_failed


:class AlreadyLoggedInError(SecureChessError)
"code = "already_logged_in


:class ValidationError(SecureChessError)
"code = "validation


:class BadStateError(SecureChessError)
"code = "bad_state


:class ServerFullError(SecureChessError)
"code = "server_full
```

src/secure_chess/common/log.py

```
Logging helper. All server output goes to `stderr` via `logging`; `stdout` is"""
""".reserved for the client UI

from __future__ import annotations

import logging
import sys


"FORMAT = "%(asctime)s %(levelname)s %(name)s | %(message)s_
configured = False_


:def get_logger(name: str, level: str = "INFO") -> logging.Logger
global _configured
:if not _configured
handler = logging.StreamHandler(sys.stderr)
handler.setFormatter(logging.Formatter(_FORMAT))
root = logging.getLogger("secure_chess")
root.addHandler(handler)
root.setLevel(level)
configured = True_
return logging.getLogger(f"secure_chess.{name}" if not name.startswith("secure_chess") else
(name
```

src/secure_chess/common/crypto.py

```
.Password hashing primitives"""

.Wraps `bcrypt` so the rest of the codebase never imports `bcrypt` directly
bcrypt` automatically generates and embeds a per-password salt in the returned`
.string, so callers don't need separate salt management
"""

from __future__ import annotations

import bcrypt


:def hash_password(plain: str) -> str
:if not isinstance(plain, str)
raise TypeError("password must be a string")
return bcrypt.hashpw(plain.encode("utf-8"), bcrypt.gensalt()).decode("ascii")


:def verify_password(plain: str, hashed: str) -> bool
:if not isinstance(plain, str) or not isinstance(hashed, str)
return False
:try
return bcrypt.checkpw(plain.encode("utf-8"), hashed.encode("ascii"))
:except (ValueError, TypeError)
return False
```

src/secure_chess/common/protocol.py

```
.JSON-Lines wire protocol over a raw TCP socket"""

Framing: one JSON object per line, UTF-8 encoded, terminated by `\\n`. The full
.`message schema lives in `specs/002-secure-chess/contracts/wire-protocol.md
"""

from __future__ import annotations

import json
import socket
from typing import Any, Dict

from secure_chess.common.errors import ConnectionClosed, ProtocolError


) = CLIENT_MESSAGE_TYPES
,"register"
,"login"
,"play_human"
,"play_ai"
,"cancel_lobby"
,"move"
,"resign"
```

```
,"quit"
(

) = SERVER_MESSAGE_TYPES
,"ok"
,"error"
,"game_started"
,"game_state"
,"game_ended"
(

MESSAGE_TYPES = CLIENT_MESSAGE_TYPES + SERVER_MESSAGE_TYPES


:def ok(**extra: Any) -> Dict[str, Any]
msg: Dict[str, Any] = {"type": "ok"}
msg.update(extra)
return msg


:def error(code: str, message: str) -> Dict[str, Any]
return {"type": "error", "code": code, "message": message}


:def send_message(sock: socket.socket, msg: Dict[str, Any]) -> None
line = (json.dumps(msg, separators=(",", ":")) + "\n").encode("utf-8")
:try
sock.sendall(line)
:except OSError as exc
raise ConnectionClosed(str(exc)) from exc


:class _LineReader
.Buffers a TCP socket and yields one JSON line per `read_line` call"""

Stored as an attribute on the `Session`/client so partial reads survive
.across calls
"""

:def __init__(self, sock: socket.socket) -> None
self._sock = sock
()self._buf = bytearray

:def read_line(self) -> bytes
:while b"\n" not in self._buf
:try
chunk = self._sock.recv(4096)
:except OSError as exc
raise ConnectionClosed(str(exc)) from exc
:if not chunk
raise ConnectionClosed("peer closed the connection")
self._buf.extend(chunk)
idx = self._buf.index(b"\n")
```

```
line = bytes(self._buf[:idx])
del self._buf[: idx + 1]
return line


:def make_reader(sock: socket.socket) -> _LineReader
return _LineReader(sock)


:def recv_message(reader: _LineReader) -> Dict[str, Any]
()line = reader.read_line
:try
msg = json.loads(line.decode("utf-8"))
:except (UnicodeDecodeError, json.JSONDecodeError) as exc
raise ProtocolError(f"malformed JSON line: {exc}") from exc
:if not isinstance(msg, dict) or "type" not in msg
raise ProtocolError("message is not a JSON object with a 'type' field")
:if msg["type"] not in MESSAGE_TYPES
raise ProtocolError(f"unknown message type: {msg['type']!r}")
return msg
```

src/secure_chess/common/user_store.py

```
.JSON-backed credential store with bcrypt-hashed passwords"""

+ The persistent file `users.json` is rewritten atomically (`write-to-tempfile
os.replace`) on every registration. A single `threading.Lock` guards every
.mutating method
"""

from __future__ import annotations

import json
import os
import re
import tempfile
import threading
from dataclasses import dataclass
from datetime import datetime, timezone
from pathlib import Path
from typing import Dict, Optional

from secure_chess.common.crypto import hash_password, verify_password
) from secure_chess.common.errors import
,DuplicateUserError
,ValidationError
,WeakPasswordError
(


USERNAME_RE = re.compile(r"^[A-Za-z0-9_-]{3,32}$")
MIN_PASSWORD_LEN = 8
```

```
dataclass@
:class Account
username: str
password_hash: str
created_at: datetime


:def _now_utc() -> datetime
return datetime.now(timezone.utc).replace(microsecond=0)


:def _validate_username(username: str) -> None
:if not isinstance(username, str)
raise ValidationError("username must be a string")
:if not USERNAME_RE.match(username)
)raise ValidationError
"username must be 3-32 chars from [A-Za-z0-9_-]"
(


:def _validate_password(password: str) -> None
:if not isinstance(password, str)
raise ValidationError("password must be a string")
:if len(password) < MIN_PASSWORD_LEN
)raise WeakPasswordError
"f"password must be at least {MIN_PASSWORD_LEN} characters
(


:class UserStore
:def __init__(self, path: str | Path) -> None
self.path = Path(path)
self.path.parent.mkdir(parents=True, exist_ok=True)
()self._lock = threading.Lock
{} = self._accounts: Dict[str, Account]
()self._load

:def _load(self) -> None
:()if not self.path.exists
()self._persist_locked
return
"{}" raw = self.path.read_text(encoding="utf-8") or
:try
data = json.loads(raw)
:except json.JSONDecodeError
{} = data
:if not isinstance(data, dict)
{} = data
{} = accounts: Dict[str, Account]
:()for username, entry in data.items
:if not isinstance(entry, dict)
continue
```

```
hashed = entry.get("password_hash")
created_raw = entry.get("created_at")
:if not isinstance(hashed, str)
continue
:try
()created = datetime.fromisoformat(created_raw) if created_raw else _now_utc
:except (TypeError, ValueError)
()created = _now_utc
)accounts[username] = Account
username=username, password_hash=hashed, created_at=created
(
self._accounts = accounts

:def _persist_locked(self) -> None
} = serial
} :a.username
,password_hash": a.password_hash"
,created_at": a.created_at.isoformat().replace("+00:00", "Z")"
{
()for a in self._accounts.values
{
text = json.dumps(serial, indent=2, sort_keys=True)
)with tempfile.NamedTemporaryFile
,mode="w", encoding="utf-8", dir=self.path.parent
"delete=False, prefix=".users.", suffix=".tmp
:as tmp (
tmp.write(text)
()tmp.flush
os.fsync(tmp.fileno())
tmp_name = tmp.name
os.replace(tmp_name, self.path)

:def __len__(self) -> int
:with self._lock
return len(self._accounts)

:def exists(self, username: str) -> bool
:with self._lock
return username in self._accounts

:def register(self, username: str, password: str) -> Account
validate_username(username)_
validate_password(password)_
hashed = hash_password(password)
:with self._lock
:if username in self._accounts
raise DuplicateUserError(f"username {username!r} already exists")
)account = Account
()username=username, password_hash=hashed, created_at=_now_utc
(
self._accounts[username] = account
()self._persist_locked
return account
```

```
:def authenticate(self, username: str, password: str) -> Optional[Account]
:if not isinstance(username, str) or not isinstance(password, str)
return None
:with self._lock
account = self._accounts.get(username)
:if account is None
return None
:if verify_password(password, account.password_hash)
return account
return None
```

src/secure_chess/common/pieces.py

```
.Chess piece types, colours, and per-piece pseudo-legal move generation"""

:This module defines
.Color`             : WHITE / BLACK with an `opposite()` helper` -
.PieceType`         : the six chess piece kinds` -
.SYMBOLS`             : single-letter symbols for ASCII rendering (uppercase = white)` -
.Piece`             : a (kind, color) pair plus pseudo-legal move generation` -

Pseudo-legal generation does NOT filter out moves that leave the mover's own king in check
that filter lives on `Board` (see `board.py`) so the same per-piece logic is reused both -
.`for legal move generation and for the king-attack scan used by `Board.is_in_check
"""

from __future__ import annotations

from dataclasses import dataclass
from enum import Enum
from typing import TYPE_CHECKING, Iterator

:if TYPE_CHECKING
from secure_chess.common.board import Board
from secure_chess.common.move import Move, Square


:class Color(Enum)
"WHITE = "white
"BLACK = "black

:"def opposite(self) -> "Color
return Color.BLACK if self is Color.WHITE else Color.WHITE


:class PieceType(Enum)
"PAWN = "pawn
"KNIGHT = "knight
"BISHOP = "bishop
"ROOK = "rook
"QUEEN = "queen
"KING = "king
```

```
} = SYMBOLS: dict[PieceType, str]
,"PieceType.PAWN: "P
,"PieceType.KNIGHT: "N
,"PieceType.BISHOP: "B
,"PieceType.ROOK: "R
,"PieceType.QUEEN: "Q
,"PieceType.KING: "K
{


) = KNIGHT_OFFSETS: tuple[tuple[int, int], ...]
,(2- ,1) ,(1- ,2) ,(1 ,2) ,(2 ,1)
,(2 ,1-) ,(1 ,2-) ,(1- ,2-) ,(2- ,1-)
(

) = KING_OFFSETS: tuple[tuple[int, int], ...]
,(1 ,1-) ,(1 ,0) ,(1 ,1) ,(0 ,1)
,(1- ,1) ,(1- ,0) ,(1- ,1-) ,(0 ,1-)
(

BISHOP_RAYS: tuple[tuple[int, int], ...] = ((1, 1), (1, -1), (-1, 1), (-1, -1))
ROOK_RAYS:   tuple[tuple[int, int], ...] = ((1, 0), (-1, 0), (0, 1), (0, -1))
QUEEN_RAYS:  tuple[tuple[int, int], ...] = BISHOP_RAYS + ROOK_RAYS


:def _in_bounds(sq: "Square") -> bool
f, r = sq
return 0 <= f < 8 and 0 <= r < 8


dataclass(frozen=True)@
:class Piece
kind: PieceType
color: Color

:def symbol(self) -> str
s = SYMBOLS[self.kind]
()return s if self.color is Color.WHITE else s.lower

:def pseudo_legal_targets(self, board: "Board", origin: "Square") -> Iterator["Move"]
from secure_chess.common.move import Move

:if self.kind is PieceType.KNIGHT
yield from _jump_moves(self, board, origin, KNIGHT_OFFSETS)
:elif self.kind is PieceType.KING
yield from _jump_moves(self, board, origin, KING_OFFSETS)
yield from _castling_moves(self, board, origin)
:elif self.kind is PieceType.BISHOP
yield from _ray_moves(self, board, origin, BISHOP_RAYS)
:elif self.kind is PieceType.ROOK
yield from _ray_moves(self, board, origin, ROOK_RAYS)
```

```
:elif self.kind is PieceType.QUEEN
yield from _ray_moves(self, board, origin, QUEEN_RAYS)
:elif self.kind is PieceType.PAWN
yield from _pawn_moves(self, board, origin)
:else
raise AssertionError(f"unknown piece kind: {self.kind}")
del Move


:def _jump_moves(piece: Piece, board: "Board", origin: "Square", offsets) -> Iterator["Move"]
from secure_chess.common.move import Move

f, r = origin
:for df, dr in offsets
target = (f + df, r + dr)
:if not _in_bounds(target)
continue
occupant = board.piece_at(target)
:if occupant is None
yield Move(origin=origin, target=target)
:elif occupant.color is not piece.color
yield Move(origin=origin, target=target, is_capture=True)


:def _ray_moves(piece: Piece, board: "Board", origin: "Square", rays) -> Iterator["Move"]
from secure_chess.common.move import Move

f, r = origin
:for df, dr in rays
step = 1
:while True
target = (f + df * step, r + dr * step)
:if not _in_bounds(target)
break
occupant = board.piece_at(target)
:if occupant is None
yield Move(origin=origin, target=target)
:else
:if occupant.color is not piece.color
yield Move(origin=origin, target=target, is_capture=True)
break
step += 1


:def _pawn_moves(piece: Piece, board: "Board", origin: "Square") -> Iterator["Move"]
from secure_chess.common.move import Move

f, r = origin
direction = 1 if piece.color is Color.WHITE else -1
start_rank = 1 if piece.color is Color.WHITE else 6
promotion_rank = 7 if piece.color is Color.WHITE else 0

one_step = (f, r + direction)
```

```
:if _in_bounds(one_step) and board.piece_at(one_step) is None
:if one_step[1] == promotion_rank
for promo in (PieceType.QUEEN, PieceType.ROOK, PieceType.BISHOP,
:(PieceType.KNIGHT
yield Move(origin=origin, target=one_step, promotion=promo)
:else
yield Move(origin=origin, target=one_step)

two_step = (f, r + 2 * direction)
:if r == start_rank and board.piece_at(two_step) is None
yield Move(origin=origin, target=two_step)

:for df in (-1, 1)
diag = (f + df, r + direction)
:if not _in_bounds(diag)
continue
occupant = board.piece_at(diag)
:if occupant is not None and occupant.color is not piece.color
:if diag[1] == promotion_rank
for promo in (PieceType.QUEEN, PieceType.ROOK, PieceType.BISHOP,
:(PieceType.KNIGHT
yield Move(origin=origin, target=diag, promotion=promo, is_capture=True)
:else
yield Move(origin=origin, target=diag, is_capture=True)
:elif occupant is None and board.en_passant_target == diag
yield Move(origin=origin, target=diag, is_capture=True, is_en_passant=True)


:def _castling_moves(piece: Piece, board: "Board", origin: "Square") -> Iterator["Move"]
from secure_chess.common.move import Move

:if piece.color is Color.WHITE
:if origin != (4, 0)
return
"king_side_flag, queen_side_flag = "K", "Q
rank = 0
:else
:if origin != (4, 7)
return
"king_side_flag, queen_side_flag = "k", "q
rank = 7

:if king_side_flag in board.castling_rights
:if board.piece_at((5, rank)) is None and board.piece_at((6, rank)) is None
\ if not board.is_square_attacked((4, rank), piece.color.opposite())
\ and not board.is_square_attacked((5, rank), piece.color.opposite())
:and not board.is_square_attacked((6, rank), piece.color.opposite())
yield Move(origin=origin, target=(6, rank), is_castle="K")
:if queen_side_flag in board.castling_rights
if (board.piece_at((1, rank)) is None
and board.piece_at((2, rank)) is None
:(and board.piece_at((3, rank)) is None
\ if not board.is_square_attacked((4, rank), piece.color.opposite())
```

```
\ and not board.is_square_attacked((3, rank), piece.color.opposite())
:and not board.is_square_attacked((2, rank), piece.color.opposite())
yield Move(origin=origin, target=(2, rank), is_castle="Q")
```

src/secure_chess/common/move.py

```
.Square coordinates, the `Move` dataclass, and coordinate-algebraic parsing"""

.'Coordinates are 0-indexed `(file, rank)` tuples where file 0 = 'a' and rank 0 = '1
`The wire notation is coordinate algebraic: `<from-square><to-square>[<promotion-piece>]
/ `examples: `e2e4`, `e7e8q`. Castling uses the king's two-square move (`e1g1 -
.(`e1c1` / `e8g8` / `e8c8`
"""

from __future__ import annotations

from dataclasses import dataclass
from typing import Literal, Optional, Tuple

from secure_chess.common.errors import MoveParseError
from secure_chess.common.pieces import PieceType


Square = Tuple[int, int]


:def to_algebraic(sq: Square) -> str
f, r = sq
:if not (0 <= f < 8 and 0 <= r < 8)
raise ValueError(f"square out of range: {sq}")
"return f"{chr(ord('a') + f)}{r + 1}


:def from_algebraic(text: str) -> Square
:"if len(text) != 2 or text[0] not in "abcdefgh" or text[1] not in "12345678
raise MoveParseError(f"invalid square: {text!r}")
return (ord(text[0]) - ord('a'), int(text[1]) - 1)


} = PROMO_MAP: dict[str, PieceType]_
,q": PieceType.QUEEN"
,r": PieceType.ROOK"
,b": PieceType.BISHOP"
,n": PieceType.KNIGHT"
{

PROMO_REV: dict[PieceType, str] = {v: k for k, v in _PROMO_MAP.items()}_


dataclass(frozen=True)@
:class Move
origin: Square
target: Square
```

```
promotion: Optional[PieceType] = None
is_capture: bool = False
is_castle: Optional[Literal["K", "Q"]] = None
is_en_passant: bool = False

classmethod@
:"def parse(cls, text: str) -> "Move
()text = text.strip().lower
:if not (4 <= len(text) <= 5)
raise MoveParseError(f"move must be 4 or 5 characters, got {text!r}")
origin = from_algebraic(text[0:2])
target = from_algebraic(text[2:4])
promotion: Optional[PieceType] = None
:if len(text) == 5
ch = text[4]
:if ch not in _PROMO_MAP
raise MoveParseError(f"invalid promotion piece: {ch!r}")
promotion = _PROMO_MAP[ch]
return cls(origin=origin, target=target, promotion=promotion)

:def to_uci(self) -> str
s = to_algebraic(self.origin) + to_algebraic(self.target)
:if self.promotion is not None
s += _PROMO_REV[self.promotion]
return s

:def matches(self, other: "Move") -> bool
""".Same origin / target / promotion (ignores flags set by `Board.apply`)"""
) return
self.origin == other.origin
and self.target == other.target
and self.promotion == other.promotion
(
```

src/secure_chess/common/board.py

```
The `Board` class - the chess position plus the auxiliary state required by"""
the rules (castling rights, en-passant target square, halfmove clock, fullmove
.(number, repetition history

.Board` is the heart of the game and the single source of truth for legality`
Every server-side move is fed through `Board.apply`, which is the only place
.that mutates the position
"""

from __future__ import annotations

from typing import Iterable, List, Optional, Tuple

from secure_chess.common.errors import IllegalMoveError
from secure_chess.common.move import Move, Square
) from secure_chess.common.pieces import
,BISHOP_RAYS
```

```
,Color
,KING_OFFSETS
,KNIGHT_OFFSETS
,Piece
,PieceType
,QUEEN_RAYS
,ROOK_RAYS
,SYMBOLS
(


} = PIECE_FROM_FEN_
,P": (PieceType.PAWN, Color.WHITE)"
,N": (PieceType.KNIGHT, Color.WHITE)"
,B": (PieceType.BISHOP, Color.WHITE)"
,R": (PieceType.ROOK, Color.WHITE)"
,Q": (PieceType.QUEEN, Color.WHITE)"
,K": (PieceType.KING, Color.WHITE)"
,p": (PieceType.PAWN, Color.BLACK)"
,n": (PieceType.KNIGHT, Color.BLACK)"
,b": (PieceType.BISHOP, Color.BLACK)"
,r": (PieceType.ROOK, Color.BLACK)"
,q": (PieceType.QUEEN, Color.BLACK)"
,k": (PieceType.KING, Color.BLACK)"
{


:class Board
""".8x8 chess board. `squares[file][rank]` is the piece on that square"""

) = __slots__
,"squares"
,"side_to_move"
,"castling_rights"
,"en_passant_target"
,"halfmove_clock"
,"fullmove_number"
,"position_history"
(

)__def __init
,self
,squares: Optional[List[List[Optional[Piece]]]] = None
,side_to_move: Color = Color.WHITE
,castling_rights: Optional[set[str]] = None
,en_passant_target: Optional[Square] = None
,halfmove_clock: int = 0
,fullmove_number: int = 1
:None <- (
self.squares: List[List[Optional[Piece]]] = squares or [[None] * 8 for _ in range(8)]
self.side_to_move: Color = side_to_move
()self.castling_rights: set[str] = castling_rights if castling_rights is not None else set
self.en_passant_target: Optional[Square] = en_passant_target
```

```
self.halfmove_clock: int = halfmove_clock
self.fullmove_number: int = fullmove_number
[] = self.position_history: List[str]

classmethod@
:"def initial(cls) -> "Board
squares: List[List[Optional[Piece]]] = [[None] * 8 for _ in range(8)]
] = back
,PieceType.ROOK, PieceType.KNIGHT, PieceType.BISHOP, PieceType.QUEEN
,PieceType.KING, PieceType.BISHOP, PieceType.KNIGHT, PieceType.ROOK
[
:for f in range(8)
squares[f][0] = Piece(back[f], Color.WHITE)
squares[f][1] = Piece(PieceType.PAWN, Color.WHITE)
squares[f][6] = Piece(PieceType.PAWN, Color.BLACK)
squares[f][7] = Piece(back[f], Color.BLACK)
)b = cls
,squares=squares
,side_to_move=Color.WHITE
,castling_rights={"K", "Q", "k", "q"}
,en_passant_target=None
,halfmove_clock=0
,fullmove_number=1
(
b.position_history.append(b._fingerprint())
return b

:def piece_at(self, sq: Square) -> Optional[Piece]
f, r = sq
return self.squares[f][r]

:def _find_king(self, color: Color) -> Square
:for f in range(8)
:for r in range(8)
p = self.squares[f][r]
:if p is not None and p.kind is PieceType.KING and p.color is color
return (f, r)
raise AssertionError(f"no {color} king on the board")

:def is_square_attacked(self, sq: Square, by_color: Color) -> bool
True if any `by_color` piece can capture onto `sq` (independent of"""
""".whose turn it is). Used both for is_in_check and for castling tests
f, r = sq
:for df, dr in KNIGHT_OFFSETS
t = (f + df, r + dr)
:if 0 <= t[0] < 8 and 0 <= t[1] < 8
p = self.squares[t[0]][t[1]]
:if p is not None and p.color is by_color and p.kind is PieceType.KNIGHT
return True

:for df, dr in KING_OFFSETS
t = (f + df, r + dr)
:if 0 <= t[0] < 8 and 0 <= t[1] < 8
```

```
p = self.squares[t[0]][t[1]]
:if p is not None and p.color is by_color and p.kind is PieceType.KING
return True

pawn_dir = -1 if by_color is Color.WHITE else 1
:for df in (-1, 1)
t = (f + df, r + pawn_dir)
:if 0 <= t[0] < 8 and 0 <= t[1] < 8
p = self.squares[t[0]][t[1]]
:if p is not None and p.color is by_color and p.kind is PieceType.PAWN
return True

:for df, dr in BISHOP_RAYS
step = 1
:while True
t = (f + df * step, r + dr * step)
:if not (0 <= t[0] < 8 and 0 <= t[1] < 8)
break
p = self.squares[t[0]][t[1]]
:if p is None
step += 1
continue
:if p.color is by_color and p.kind in (PieceType.BISHOP, PieceType.QUEEN)
return True
break

:for df, dr in ROOK_RAYS
step = 1
:while True
t = (f + df * step, r + dr * step)
:if not (0 <= t[0] < 8 and 0 <= t[1] < 8)
break
p = self.squares[t[0]][t[1]]
:if p is None
step += 1
continue
:if p.color is by_color and p.kind in (PieceType.ROOK, PieceType.QUEEN)
return True
break

return False

:def is_in_check(self, color: Color) -> bool
return self.is_square_attacked(self._find_king(color), color.opposite())

:def _pseudo_legal_moves(self, color: Color) -> Iterable[Move]
:for f in range(8)
:for r in range(8)
p = self.squares[f][r]
:if p is None or p.color is not color
continue
yield from p.pseudo_legal_targets(self, (f, r))
```

```
:def legal_moves(self, color: Optional[Color] = None) -> List[Move]
side = color or self.side_to_move
[] = result: List[Move]
:for move in list(self._pseudo_legal_moves(side))
()snapshot = self._snapshot
:try
self._apply_unchecked(move)
in_check_after = self.is_in_check(side)
:finally
self._restore(snapshot)
:if not in_check_after
result.append(move)
return result

:def is_legal(self, move: Move) -> bool
:for legal in self.legal_moves(self.side_to_move)
:if legal.matches(move)
return True
return False

:def apply(self, move: Move) -> Move
Validate `move` and mutate the board. Returns the resolved move with"""
""".flags (capture, castle, en-passant, promotion) populated
resolved: Optional[Move] = None
:for legal in self.legal_moves(self.side_to_move)
:if legal.matches(move)
resolved = legal
break
:if resolved is None
raise IllegalMoveError(f"move {move.to_uci()!r} is not legal in the current position")
self._apply_unchecked(resolved)
return resolved

:def _snapshot(self)
) return
,[row[:] for row in self.squares]
,self.side_to_move
,set(self.castling_rights)
,self.en_passant_target
,self.halfmove_clock
,self.fullmove_number
,list(self.position_history)
(

:def _restore(self, snap) -> None
)
,self.squares
,self.side_to_move
,self.castling_rights
,self.en_passant_target
,self.halfmove_clock
,self.fullmove_number
,self.position_history
```

```
) = (
,[row[:] for row in snap[0]]
,snap[1]
,set(snap[2])
,snap[3]
,snap[4]
,snap[5]
,list(snap[6])
(

:def _apply_unchecked(self, move: Move) -> None
origin = move.origin
target = move.target
piece = self.squares[origin[0]][origin[1]]
"assert piece is not None, f"no piece on origin square {origin}

captured = self.squares[target[0]][target[1]]
is_pawn_move = piece.kind is PieceType.PAWN
is_capture = captured is not None or move.is_en_passant

self.squares[origin[0]][origin[1]] = None

:if move.is_en_passant
cap_rank = target[1] + (-1 if piece.color is Color.WHITE else 1)
self.squares[target[0]][cap_rank] = None
self.squares[target[0]][target[1]] = piece
:elif move.promotion is not None
self.squares[target[0]][target[1]] = Piece(move.promotion, piece.color)
:else
self.squares[target[0]][target[1]] = piece

:"if move.is_castle == "K
rank = 0 if piece.color is Color.WHITE else 7
rook = self.squares[7][rank]
self.squares[7][rank] = None
self.squares[5][rank] = rook
:"elif move.is_castle == "Q
rank = 0 if piece.color is Color.WHITE else 7
rook = self.squares[0][rank]
self.squares[0][rank] = None
self.squares[3][rank] = rook

:if piece.kind is PieceType.KING
:if piece.color is Color.WHITE
self.castling_rights.discard("K")
self.castling_rights.discard("Q")
:else
self.castling_rights.discard("k")
self.castling_rights.discard("q")
:if piece.kind is PieceType.ROOK
:if origin == (0, 0)
self.castling_rights.discard("Q")
:elif origin == (7, 0)
```

```
self.castling_rights.discard("K")
:elif origin == (0, 7)
self.castling_rights.discard("q")
:elif origin == (7, 7)
self.castling_rights.discard("k")
:if target == (0, 0)
self.castling_rights.discard("Q")
:elif target == (7, 0)
self.castling_rights.discard("K")
:elif target == (0, 7)
self.castling_rights.discard("q")
:elif target == (7, 7)
self.castling_rights.discard("k")

:if is_pawn_move and abs(target[1] - origin[1]) == 2
self.en_passant_target = (origin[0], (origin[1] + target[1]) // 2)
:else
self.en_passant_target = None

:if is_pawn_move or is_capture
self.halfmove_clock = 0
:else
self.halfmove_clock += 1

:if self.side_to_move is Color.BLACK
self.fullmove_number += 1
()self.side_to_move = self.side_to_move.opposite

self.position_history.append(self._fingerprint())

:def is_checkmate(self) -> bool
return self.is_in_check(self.side_to_move) and not self.legal_moves(self.side_to_move)

:def is_stalemate(self) -> bool
return not self.is_in_check(self.side_to_move) and not self.legal_moves(self.side_to_move)

:def is_fifty_move_draw(self) -> bool
return self.halfmove_clock >= 100

:def is_threefold_repetition(self) -> bool
:if not self.position_history
return False
last = self.position_history[-1]
return self.position_history.count(last) >= 3

:def _fingerprint(self) -> str
\ return self._placement_only() + " " + ("w" if self.side_to_move is Color.WHITE else "b")
()self._castling_string() + " " + self._ep_string + " " +

:def _placement_only(self) -> str
[] = rows: List[str]
:for rank in range(7, -1, -1)
"" = row
```

```
empty = 0
:for f in range(8)
p = self.squares[f][rank]
:if p is None
empty += 1
:else
:if empty
row += str(empty)
empty = 0
sym = SYMBOLS[p.kind]
()row += sym if p.color is Color.WHITE else sym.lower
:if empty
row += str(empty)
rows.append(row)
return "/".join(rows)

:def _castling_string(self) -> str
:if not self.castling_rights
"-" return
"order = "KQkq
return "".join(ch for ch in order if ch in self.castling_rights)

:def _ep_string(self) -> str
from secure_chess.common.move import to_algebraic
"-" return to_algebraic(self.en_passant_target) if self.en_passant_target else

:def to_fen_short(self) -> str
return f"{self._placement_only()} {'w' if self.side_to_move is Color.WHITE else 'b'}
"{self._castling_string()} {self._ep_string()}

classmethod@
:"def from_fen_short(cls, fen: str) -> "Board
from secure_chess.common.move import from_algebraic

()parts = fen.strip().split
:if len(parts) < 4
raise ValueError(f"invalid short FEN: {fen!r}")
placement, side, castling, ep = parts[0], parts[1], parts[2], parts[3]
squares: List[List[Optional[Piece]]] = [[None] * 8 for _ in range(8)]
("/")rows = placement.split
:if len(rows) != 8
raise ValueError(f"invalid placement: {placement!r}")
:for rank_idx, row in enumerate(rows)
rank = 7 - rank_idx
f = 0
:for ch in row
:()if ch.isdigit
f += int(ch)
:else
kind, color = _PIECE_FROM_FEN[ch]
squares[f][rank] = Piece(kind, color)
f += 1
)b = cls
```

```
,squares=squares
,side_to_move=Color.WHITE if side == "w" else Color.BLACK
,castling_rights=set() if castling == "-" else set(castling)
,en_passant_target=None if ep == "-" else from_algebraic(ep)
(
b.position_history.append(b._fingerprint())
return b
```

src/secure_chess/common/game.py

```
""".`Game` and `Result` - the match between two sessions sharing a `Board`"""

from __future__ import annotations

import uuid
from dataclasses import dataclass
from datetime import datetime, timezone
from typing import TYPE_CHECKING, Optional

from secure_chess.common.board import Board
from secure_chess.common.errors import IllegalMoveError, NotYourTurnError
from secure_chess.common.move import Move
from secure_chess.common.pieces import Color


:if TYPE_CHECKING
from secure_chess.server.session import Session


dataclass(frozen=True)@
:class Result
"outcome: str    # "white_wins" | "black_wins" | "draw
"reason: str     # "checkmate" | "stalemate" | "fifty-move" | "threefold" | "resignation" | "disconnect

classmethod@
:"def white_wins(cls, reason: str) -> "Result
return cls("white_wins", reason)

classmethod@
:"def black_wins(cls, reason: str) -> "Result
return cls("black_wins", reason)

classmethod@
:"def draw(cls, reason: str) -> "Result
return cls("draw", reason)


:class Game
:def __init__(self, white: "Session", black: "Session") -> None
self.id: str = uuid.uuid4().hex
self.white: "Session" = white
self.black: "Session" = black
()self.board: Board = Board.initial
```

```
self.result: Optional[Result] = None
self.created_at: datetime = datetime.now(timezone.utc)
self.last_move_uci: Optional[str] = None

:"def current_session(self) -> "Session
return self.white if self.board.side_to_move is Color.WHITE else self.black

:def color_of(self, session: "Session") -> Optional[Color]
:if session is self.white
return Color.WHITE
:if session is self.black
return Color.BLACK
return None

:def submit_move(self, session: "Session", move: Move) -> Move
:if self.result is not None
raise IllegalMoveError("game has already ended")
mover_color = self.color_of(session)
:if mover_color is None
raise NotYourTurnError("session is not a participant in this game")
:if mover_color is not self.board.side_to_move
raise NotYourTurnError("it is not your turn")

resolved = self.board.apply(move)
()self.last_move_uci = resolved.to_uci

:()if self.board.is_checkmate
winner_color = mover_color
) = self.result
Result.white_wins("checkmate") if winner_color is Color.WHITE
else Result.black_wins("checkmate")
(
:()elif self.board.is_stalemate
self.result = Result.draw("stalemate")
:()elif self.board.is_threefold_repetition
self.result = Result.draw("threefold")
:()elif self.board.is_fifty_move_draw
self.result = Result.draw("fifty-move")
return resolved

:def resign(self, session: "Session") -> None
:if self.result is not None
return
color = self.color_of(session)
:if color is Color.WHITE
self.result = Result.black_wins("resignation")
:else
self.result = Result.white_wins("resignation")

:def handle_disconnect(self, session: "Session") -> None
:if self.result is not None
return
color = self.color_of(session)
```

```
:if color is Color.WHITE
self.result = Result.black_wins("disconnect")
:elif color is Color.BLACK
self.result = Result.white_wins("disconnect")
```

src/secure_chess/common/ai.py

```
.Simple alpha-beta minimax chess AI with material-balance evaluation"""

.This is the BONUS AI opponent (US4). Stronger play is out of scope
"""

from __future__ import annotations

import math
import random
from typing import Optional

from secure_chess.common.board import Board
from secure_chess.common.move import Move
from secure_chess.common.pieces import Color, PieceType


} = PIECE_VALUE
,PieceType.PAWN: 100
,PieceType.KNIGHT: 300
,PieceType.BISHOP: 300
,PieceType.ROOK: 500
,PieceType.QUEEN: 900
,PieceType.KING: 0
{

CHECKMATE_SCORE = 1_000_000


:def evaluate(board: Board) -> int
""".Material-balance from White's perspective, in centipawns"""
score = 0
:for f in range(8)
:for r in range(8)
p = board.squares[f][r]
:if p is None
continue
val = PIECE_VALUE[p.kind]
score += val if p.color is Color.WHITE else -val
return score


:class AIPlayer
""".Pick a legal move via alpha-beta minimax to a fixed depth"""

:def choose_move(self, board: Board, depth: int = 3) -> Move
depth = max(1, min(4, int(depth)))
```

```
side = board.side_to_move
legal = board.legal_moves(side)
:if not legal
raise RuntimeError("AI asked to move in a terminal position")

best_score: Optional[int] = None
[] = best_moves: list[Move]
alpha, beta = -math.inf, math.inf

:for move in legal
()snap = board._snapshot
:try
board._apply_unchecked(move)
)score = -self._negamax
,board, depth - 1, -beta, -alpha
()perspective=side.opposite
(
:finally
board._restore(snap)

:if best_score is None or score > best_score
best_score = score
best_moves = [move]
:elif score == best_score
best_moves.append(move)
:if score > alpha
alpha = score
"assert best_moves, "negamax produced no candidate
return random.choice(best_moves)

:def _negamax(self, board: Board, depth: int, alpha: float, beta: float, perspective: Color) -> int
:if depth == 0
return self._perspective_eval(board, perspective)
legal = board.legal_moves(board.side_to_move)
:if not legal
:if board.is_in_check(board.side_to_move)
return -CHECKMATE_SCORE + (10 - depth)
return 0

best = -math.inf
:for move in legal
()snap = board._snapshot
:try
board._apply_unchecked(move)
)score = -self._negamax
,board, depth - 1, -beta, -alpha
()perspective=perspective.opposite
(
:finally
board._restore(snap)
:if score > best
best = score
:if best > alpha
```

```
alpha = best
:if alpha >= beta
break
return int(best)

staticmethod@
:def _perspective_eval(board: Board, perspective: Color) -> int
white_score = evaluate(board)
return white_score if perspective is Color.WHITE else -white_score
```

src/secure_chess/server/__init__.py

```
""".TCP server: accept loop, per-client session threads, lobby, game registry"""
```

src/secure_chess/server/__main__.py

```
""".`CLI entry point: `python -m secure_chess.server"""

from __future__ import annotations

import argparse
import sys
import threading

from secure_chess.common.log import get_logger
from secure_chess.server.server import ChessServer


:def main(argv: list[str] | None = None) -> int
parser = argparse.ArgumentParser(prog="secure-chess-server")
parser.add_argument("--host", default="127.0.0.1")
parser.add_argument("--port", type=int, default=5050)
parser.add_argument("--data-dir", default="data")
parser.add_argument("--ai-depth", type=int, default=3)
parser.add_argument("--max-clients", type=int, default=32)
,"parser.add_argument("--log-level", default="INFO
(choices=["DEBUG", "INFO", "WARNING", "ERROR"]
args = parser.parse_args(argv)

get_logger("main", level=args.log_level)

)server = ChessServer
,host=args.host
,port=args.port
,data_dir=args.data_dir
,ai_depth=args.ai_depth
,max_clients=args.max_clients
(
()server.start

()ev = threading.Event
:try
()ev.wait
:except KeyboardInterrupt
```

```
pass
:finally
()server.stop
return 0


if __name__ == "__main__":  # pragma: no cover
sys.exit(main())
```

src/secure_chess/server/lobby.py

```
.Server-side match-making (`Lobby`) and active-game tracking (`GameRegistry`)"""

Both are thread-safe so multiple session threads can call them concurrently
.this is what enables the parallel-games bonus (US3) -
"""

from __future__ import annotations

import threading
from typing import TYPE_CHECKING, Dict, List, Optional, Tuple

from secure_chess.common.game import Game, Result
from secure_chess.common.log import get_logger
from secure_chess.common.pieces import Color


:if TYPE_CHECKING
from secure_chess.server.session import Session


log = get_logger("lobby")


:class Lobby
:def __init__(self) -> None
[] = self._queue: List["Session"]
()self._lock = threading.Lock

:def enqueue(self, session: "Session") -> Optional[Tuple["Session", "Session"]]
:with self._lock
self._queue.append(session)
:if len(self._queue) >= 2
white = self._queue.pop(0)
black = self._queue.pop(0)
return white, black
return None

:def remove(self, session: "Session") -> None
:with self._lock
:try
self._queue.remove(session)
:except ValueError
```

```
pass

:def __len__(self) -> int
:with self._lock
return len(self._queue)


:class GameRegistry
:def __init__(self) -> None
{} = self._games: Dict[str, Game]
()self._lock = threading.Lock

:def create(self, white: "Session", black: "Session") -> Game
from secure_chess.server.session import SessionState

game = Game(white=white, black=black)
:with self._lock
self._games[game.id] = game

white.current_game = game
black.current_game = game
white.state = SessionState.IN_GAME
black.state = SessionState.IN_GAME

()board_fen = game.board.to_fen_short
})white.send
,"type": "game_started"
,game_id": game.id"
,"you_are": "white"
,opponent": black.username"
,board": board_fen"
,"to_move": "white"
({
})black.send
,"type": "game_started"
,game_id": game.id"
,"you_are": "black"
,opponent": white.username"
,board": board_fen"
,"to_move": "white"
({
,"log.info("game %s started: %s (white) vs %s (black)
(game.id[:8], white.username, black.username
return game

:def broadcast_state(self, game: Game, last_move: str) -> None
} = msg
,"type": "game_state"
,game_id": game.id"
,last_move": last_move"
,()board": game.board.to_fen_short"
,"to_move": "white" if game.board.side_to_move is Color.WHITE else "black"
,in_check": game.board.is_in_check(game.board.side_to_move)"
```

```
{
game.white.send(msg)
game.black.send(msg)

:def end(self, game: Game) -> None
from secure_chess.server.session import SessionState

:with self._lock
self._games.pop(game.id, None)

:if game.result is None
game.result = Result.draw("aborted")

} = msg
,"type": "game_ended"
,game_id": game.id"
,result": game.result.outcome"
,reason": game.result.reason"
{
game.white.send(msg)
game.black.send(msg)
game.white.current_game = None
game.black.current_game = None
:if game.white.state is SessionState.IN_GAME
game.white.state = SessionState.AUTHENTICATED
:if game.black.state is SessionState.IN_GAME
game.black.state = SessionState.AUTHENTICATED
,"log.info("game %s ended: %s (%s)
(game.id[:8], game.result.outcome, game.result.reason

:def drive_ai(self, ai_session) -> None
""".If `ai_session` is the side to move in its game, make the AI move now"""
from secure_chess.common.ai import AIPlayer

game = ai_session.current_game
:if game is None or game.result is not None
return
:if game.current_session() is not ai_session
return
move = AIPlayer().choose_move(game.board, depth=ai_session.depth)
resolved = game.submit_move(ai_session, move)
()uci = resolved.to_uci
:if game.result is None
self.broadcast_state(game, last_move=uci)
:else
self.end(game)

:def active_games(self) -> List[Game]
:with self._lock
return list(self._games.values())
```

src/secure_chess/server/session.py

```
.Per-client server-side `Session` and the `SessionState` machine"""

Each accepted TCP connection runs in its own thread driving `Session.run()`. The
,handlers `handle_<type>` are added incrementally per user story (auth in US1
.(game in US2, AI in US4
"""

from __future__ import annotations

import socket
import threading
import uuid
from enum import Enum
from typing import TYPE_CHECKING, Any, Callable, Dict, Optional

from secure_chess.common import protocol
) from secure_chess.common.errors import
,AlreadyLoggedInError
,AuthenticationError
,BadStateError
,ConnectionClosed
,DuplicateUserError
,IllegalMoveError
,MoveParseError
,NotYourTurnError
,ProtocolError
,SecureChessError
,ValidationError
,WeakPasswordError
(
from secure_chess.common.log import get_logger


:if TYPE_CHECKING
from secure_chess.common.user_store import Account, UserStore
from secure_chess.common.game import Game
from secure_chess.server.lobby import GameRegistry, Lobby


log = get_logger("session")


:class SessionState(Enum)
"ANONYMOUS = "anonymous
"AUTHENTICATED = "authenticated
"IN_LOBBY = "in_lobby
"IN_GAME = "in_game


:class Session
""".One connected client. Owns its socket; survives as long as the TCP connection"""
```

```
)__def __init
,self
,sock: socket.socket
,peer: tuple[str, int]
,"user_store: "UserStore
,"lobby: "Lobby
,"registry: "GameRegistry
,active_logins: Dict[str, "Session"]
,active_logins_lock: threading.Lock
,ai_depth: int = 3
:None <- (
self.id: str = uuid.uuid4().hex
self.socket: socket.socket = sock
self.peer = peer
self.account: Optional["Account"] = None
self.current_game: Optional["Game"] = None
self.state: SessionState = SessionState.ANONYMOUS

self._reader = protocol.make_reader(sock)
()self._write_lock = threading.Lock
self._closed = False

self.user_store = user_store
self.lobby = lobby
self.registry = registry
self._active_logins = active_logins
self._active_logins_lock = active_logins_lock
self.ai_depth = ai_depth

property@
:def username(self) -> str
"<return self.account.username if self.account else "<anonymous

:def send(self, msg: Dict[str, Any]) -> None
:if self._closed
return
:with self._write_lock
:try
protocol.send_message(self.socket, msg)
:except ConnectionClosed
self._closed = True

:def close(self) -> None
:if self._closed
return
self._closed = True
:try
self.socket.shutdown(socket.SHUT_RDWR)
:except OSError
pass
:try
()self.socket.close
```

```
:except OSError
pass

:def run(self) -> None
log.info("[%s] connected from %s:%d", self.id[:8], *self.peer)
:try
:while not self._closed
:try
msg = protocol.recv_message(self._reader)
:except ConnectionClosed
log.info("[%s] disconnected (%s)", self.id[:8], self.username)
break
:except ProtocolError as exc
self.send(protocol.error(exc.code, str(exc)))
continue
:try
self._dispatch(msg)
:except SecureChessError as exc
self.send(protocol.error(exc.code, str(exc)))
except Exception as exc:  # pragma: no cover - defensive
log.exception("[%s] unexpected error: %s", self.id[:8], exc)
self.send(protocol.error("internal_error", "internal server error"))
:finally
()self._on_disconnect
()self.close

:def _dispatch(self, msg: Dict[str, Any]) -> None
kind = msg["type"]
handler: Optional[Callable[[Dict[str, Any]], None]] = getattr(self, f"handle_{kind}", None)
:if handler is None
raise BadStateError(f"command {kind!r} is not legal in state {self.state.value!r}")
handler(msg)

:def _on_disconnect(self) -> None
""".Clean-up when the TCP socket goes away. Concrete behaviour added in US2"""
:if self.state is SessionState.IN_LOBBY
self.lobby.remove(self)
:elif self.state is SessionState.IN_GAME and self.current_game is not None
self.current_game.handle_disconnect(self)
self.registry.end(self.current_game)
:if self.account is not None
:with self._active_logins_lock
:if self._active_logins.get(self.account.username) is self
del self._active_logins[self.account.username]

:def handle_register(self, msg: Dict[str, Any]) -> None
:if self.state is not SessionState.ANONYMOUS
raise BadStateError("already authenticated")
username = msg.get("username", "")
password = msg.get("password", "")
:if not isinstance(username, str) or not isinstance(password, str)
raise ValidationError("username and password must be strings")
:try
```

```
account = self.user_store.register(username, password)
:except (DuplicateUserError, WeakPasswordError, ValidationError)
raise
self._login_as(account)
self.send(protocol.ok())

:def handle_login(self, msg: Dict[str, Any]) -> None
:if self.state is not SessionState.ANONYMOUS
raise BadStateError("already authenticated")
username = msg.get("username", "")
password = msg.get("password", "")
:if not isinstance(username, str) or not isinstance(password, str)
raise ValidationError("username and password must be strings")
account = self.user_store.authenticate(username, password)
:if account is None
raise AuthenticationError("invalid username or password")
self._login_as(account)
self.send(protocol.ok())

:def _login_as(self, account: "Account") -> None
:with self._active_logins_lock
:if account.username in self._active_logins
raise AlreadyLoggedInError(f"account {account.username!r} is already logged in")
self._active_logins[account.username] = self
self.account = account
self.state = SessionState.AUTHENTICATED
log.info("[%s] authenticated as %s", self.id[:8], account.username)

:def handle_quit(self, msg: Dict[str, Any]) -> None
self.send(protocol.ok())
self._closed = True

:def handle_play_human(self, msg: Dict[str, Any]) -> None
:if self.state is not SessionState.AUTHENTICATED
raise BadStateError("must be authenticated and not already in a game/lobby")
self.state = SessionState.IN_LOBBY
self.send(protocol.ok())
pair = self.lobby.enqueue(self)
:if pair is not None
white, black = pair
self.registry.create(white, black)

:def handle_cancel_lobby(self, msg: Dict[str, Any]) -> None
:if self.state is not SessionState.IN_LOBBY
raise BadStateError("not currently waiting in a lobby")
self.lobby.remove(self)
self.state = SessionState.AUTHENTICATED
self.send(protocol.ok())

:def handle_move(self, msg: Dict[str, Any]) -> None
:if self.state is not SessionState.IN_GAME or self.current_game is None
raise BadStateError("no active game")
uci = msg.get("uci", "")
```

```
:if not isinstance(uci, str)
raise ValidationError("'uci' must be a string")
from secure_chess.common.move import Move

move = Move.parse(uci)
game = self.current_game
:try
game.submit_move(self, move)
:except (IllegalMoveError, NotYourTurnError, MoveParseError)
raise
self.send(protocol.ok())
:if game.result is None
self.registry.broadcast_state(game, last_move=uci)
()next_session = game.current_session
:if isinstance(next_session, AISession)
self.registry.drive_ai(next_session)
:else
self.registry.end(game)

:def handle_resign(self, msg: Dict[str, Any]) -> None
:if self.state is not SessionState.IN_GAME or self.current_game is None
raise BadStateError("no active game to resign")
game = self.current_game
game.resign(self)
self.send(protocol.ok())
self.registry.end(game)

:def handle_play_ai(self, msg: Dict[str, Any]) -> None
:if self.state is not SessionState.AUTHENTICATED
raise BadStateError("must be authenticated and not already in a game/lobby")
from secure_chess.common.pieces import Color

requested_color = msg.get("color", "white")
:if requested_color not in ("white", "black")
raise ValidationError("'color' must be 'white' or 'black'")
depth_raw = msg.get("depth", self.ai_depth)
:try
depth = int(depth_raw)
:except (TypeError, ValueError)
raise ValidationError("'depth' must be an integer")
depth = max(1, min(4, depth))

ai_session = AISession(depth=depth, registry=self.registry)
human_color = Color.WHITE if requested_color == "white" else Color.BLACK
:if human_color is Color.WHITE
white, black = self, ai_session
:else
white, black = ai_session, self
self.registry.create(white, black)
:if isinstance(white, AISession)
self.registry.drive_ai(white)
```

```
:class AISession
.In-memory session impersonating an AI opponent"""

Implements the slice of the `Session` interface that `Game` and
,`GameRegistry` actually consume: `send`, `account`, `state`, `current_game`
.and a `username`. Its `send` is a no-op because there is no socket
"""

:def __init__(self, depth: int, registry: "GameRegistry") -> None
self.id: str = uuid.uuid4().hex
self.account = _AIAccount(username=f"AI(depth={depth})")
self.current_game: Optional["Game"] = None
self.state: SessionState = SessionState.IN_GAME
self.depth = depth
self.registry = registry

property@
:def username(self) -> str
return self.account.username

def send(self, msg: Dict[str, Any]) -> None:  # pragma: no cover - intentionally inert
return None

:def close(self) -> None
return None


:class _AIAccount
Tiny stand-in for an `Account` so `Game`/`GameRegistry` can treat the AI"""
""".`like any other player when reading `session.account.username

:def __init__(self, username: str) -> None
self.username = username
```

src/secure_chess/server/server.py

```
.TCP accept loop for the secure-chess server"""

/ `ChessServer` owns the listening socket and one shared `UserStore` / `Lobby`
GameRegistry`. Every accepted connection becomes a `Session` running on its`
.own daemon thread
"""

from __future__ import annotations

import socket
import threading
from pathlib import Path
from typing import Dict, Optional

from secure_chess.common import protocol
from secure_chess.common.errors import ServerFullError
from secure_chess.common.log import get_logger
```

```
from secure_chess.common.user_store import UserStore
from secure_chess.server.lobby import GameRegistry, Lobby
from secure_chess.server.session import Session


log = get_logger("server")


:class ChessServer
)__def __init
,self
,"host: str = "127.0.0.1
,port: int = 5050
,"data_dir: str | Path = "data
,ai_depth: int = 3
,max_clients: int = 32
:None <- (
self.host = host
self.port = port
self.data_dir = Path(data_dir)
self.data_dir.mkdir(parents=True, exist_ok=True)

self.user_store = UserStore(self.data_dir / "users.json")
()self.lobby = Lobby
()self.registry = GameRegistry
self.ai_depth = ai_depth
self.max_clients = max_clients

self._sock: Optional[socket.socket] = None
()self._stop_event = threading.Event
[] = self._sessions: list[Session]
()self._sessions_lock = threading.Lock
{} = self._active_logins: Dict[str, Session]
()self._active_logins_lock = threading.Lock
self._accept_thread: Optional[threading.Thread] = None
self.bound_address: tuple[str, int] | None = None

:def start(self) -> tuple[str, int]
self._sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
self._sock.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
self._sock.bind((self.host, self.port))
self._sock.listen(self.max_clients)
()self.bound_address = self._sock.getsockname
log.info("listening on %s:%d", *self.bound_address)
)log.info
credential store: %s (%d accounts)", self.user_store.path, len(self.user_store)"
(
self._accept_thread = threading.Thread(target=self._accept_loop, daemon=True)
()self._accept_thread.start
return self.bound_address

:def _accept_loop(self) -> None
assert self._sock is not None
```

```
self._sock.settimeout(0.5)
:()while not self._stop_event.is_set
:try
()client_sock, peer = self._sock.accept
:except socket.timeout
continue
:except OSError
break

:with self._sessions_lock
live = [s for s in self._sessions if not getattr(s, "_closed", False)]
self._sessions = live
:if len(live) >= self.max_clients
log.warning("refusing connection from %s:%d - server full", *peer)
:try
)protocol.send_message
,client_sock
,protocol.error(ServerFullError.code, "server is at capacity")
(
:except OSError
pass
()client_sock.close
continue

)session = Session
,sock=client_sock
,peer=peer
,user_store=self.user_store
,lobby=self.lobby
,registry=self.registry
,active_logins=self._active_logins
,active_logins_lock=self._active_logins_lock
,ai_depth=self.ai_depth
(
:with self._sessions_lock
self._sessions.append(session)
thread = threading.Thread(target=session.run, daemon=True, name=f"sess-{session.id[:6]}")
()thread.start

:def stop(self) -> None
()self._stop_event.set
:if self._sock is not None
:try
()self._sock.close
:except OSError
pass
:if self._accept_thread is not None
self._accept_thread.join(timeout=2.0)
:with self._sessions_lock
:for s in self._sessions
()s.close
[] = self._sessions
log.info("server stopped")
```

src/secure_chess/client/__init__.py

```
""".Console client: REPL prompt, ASCII board renderer, push-message handlers"""
```

src/secure_chess/client/__main__.py

```
.`Client entry point: `python -m secure_chess.client"""

By default this launches the Tkinter GUI client. Pass `--cli` to run the
text-mode REPL client instead — useful for headless smoke tests and for users
.on systems without a display
"""

from __future__ import annotations

import argparse
import sys


:def main(argv: list[str] | None = None) -> int
parser = argparse.ArgumentParser(prog="secure-chess-client")
parser.add_argument("--host", default="127.0.0.1")
parser.add_argument("--port", type=int, default=5050)
)parser.add_argument
,"cli--"
,"action="store_true
,"help="Use the text-mode REPL instead of the Tkinter GUI
(
args = parser.parse_args(argv)

:if args.cli
from secure_chess.client.cli import run_cli

return run_cli(args.host, args.port)

:try
from secure_chess.client.gui import run_gui
:except ImportError as exc
)print
" .f"GUI is unavailable ({exc}); falling back to CLI
,".f"Re-run with --cli to suppress this warning
,file=sys.stderr
(
from secure_chess.client.cli import run_cli

return run_cli(args.host, args.port)

return run_gui(args.host, args.port)


if __name__ == "__main__":  # pragma: no cover
sys.exit(main())
```

src/secure_chess/client/cli.py

```
.Console UI for the secure-chess client"""

A foreground reader thread continuously prints push messages while the main
.thread reads commands from stdin and forwards them to the server
"""

from __future__ import annotations

import shlex
import socket
import sys
import threading
from typing import Any, Dict, Optional

from secure_chess.common import protocol
from secure_chess.common.errors import ConnectionClosed, ProtocolError


} = PIECE_TO_GLYPH
,"K": "K", "Q": "Q", "R": "R", "B": "B", "N": "N", "P": "P"
,"k": "k", "q": "q", "r": "r", "b": "b", "n": "n", "p": "p"
{


:def render_board(fen_short: str) -> str
""".Render the first FEN field as an 8-rank ASCII grid"""
placement = fen_short.split()[0]
[] = lines: list[str]
:for rank_idx, row in enumerate(placement.split("/"))
rank_label = 8 - rank_idx
[] = cells: list[str]
:for ch in row
:()if ch.isdigit
cells.extend(["."] * int(ch))
:else
cells.append(PIECE_TO_GLYPH.get(ch, ch))
lines.append(f"{rank_label} {' '.join(cells)}")
lines.append("  a b c d e f g h")
return "\n".join(lines)


:class ClientConnection
:def __init__(self, host: str, port: int) -> None
self._sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
self._sock.connect((host, port))
self._reader = protocol.make_reader(self._sock)
()self._send_lock = threading.Lock
()self._stop = threading.Event
self.you_are: Optional[str] = None
self.opponent: Optional[str] = None
self.last_board: Optional[str] = None
```

```
self.game_id: Optional[str] = None

:def send(self, msg: Dict[str, Any]) -> None
:with self._send_lock
protocol.send_message(self._sock, msg)

:def reader_loop(self) -> None
:()while not self._stop.is_set
:try
msg = protocol.recv_message(self._reader)
:except ConnectionClosed
print("\n[connection closed by server]")
()self._stop.set
return
:except ProtocolError as exc
print(f"\n[protocol error from server: {exc}]")
continue
self._on_message(msg)

:def _on_message(self, msg: Dict[str, Any]) -> None
t = msg["type"]
:"if t == "ok
extras = {k: v for k, v in msg.items() if k != "type"}
:if extras
print(f"ok  {extras}")
:else
print("ok")
:"elif t == "error
print(f"error: {msg.get('code')} - {msg.get('message')}")
:"elif t == "game_started
self.you_are = msg.get("you_are")
self.opponent = msg.get("opponent")
self.game_id = msg.get("game_id")
self.last_board = msg.get("board")
()print
print("== game started ==")
print(f"you are: {self.you_are}   opponent: {self.opponent}   to move: {msg.get('to_move')}")
:if self.last_board
print(render_board(self.last_board))
:"elif t == "game_state
self.last_board = msg.get("board")
()print
"print(f"last move: {msg.get('last_move')}   to move: {msg.get('to_move')}
(("" CHECK" if msg.get("in_check") else   ") +
:if self.last_board
print(render_board(self.last_board))
:"elif t == "game_ended
()print
print(f"== game ended == result: {msg.get('result')}   reason: {msg.get('reason')}")
self.you_are = None
self.opponent = None
self.last_board = None
self.game_id = None
```

```
else:  # pragma: no cover - server should only send known types
print(f"[unknown push: {msg}]")

:def stop(self) -> None
()self._stop.set
:try
self._sock.shutdown(socket.SHUT_RDWR)
:except OSError
pass
()self._sock.close


:def _split_args(line: str) -> list[str]
:try
return shlex.split(line)
:except ValueError
()return line.split


:def run_cli(host: str, port: int) -> int
print(f"connected to {host}:{port}")
conn = ClientConnection(host, port)
reader_thread = threading.Thread(target=conn.reader_loop, daemon=True, name="client-
("reader
()reader_thread.start
:try
:()while not conn._stop.is_set
:try
()line = input("> ").strip
:except (EOFError, KeyboardInterrupt)
break
:if not line
continue
parts = _split_args(line)
cmd, *args = parts
:if cmd in {"quit", "exit"}
conn.send({"type": "quit"})
break
:"if cmd == "board
:if conn.last_board
print(render_board(conn.last_board))
:else
print("(no board to show)")
continue
:try
msg = _build_message(cmd, args)
:except ValueError as exc
print(f"error: {exc}")
continue
:try
conn.send(msg)
:except (OSError, ConnectionClosed) as exc
print(f"[send failed: {exc}]")
```

```
break
:finally
()conn.stop
return 0


:def _build_message(cmd: str, args: list[str]) -> Dict[str, Any]
:"if cmd == "register
:if len(args) != 2
raise ValueError("usage: register <username> <password>")
return {"type": "register", "username": args[0], "password": args[1]}
:"if cmd == "login
:if len(args) != 2
raise ValueError("usage: login <username> <password>")
return {"type": "login", "username": args[0], "password": args[1]}
:"if cmd == "play_human
return {"type": "play_human"}
:"if cmd == "cancel_lobby
return {"type": "cancel_lobby"}
:"if cmd == "play_ai
msg: Dict[str, Any] = {"type": "play_ai"}
:if len(args) >= 1
:if args[0] not in ("white", "black")
raise ValueError("color must be 'white' or 'black'")
msg["color"] = args[0]
:if len(args) >= 2
:try
msg["depth"] = int(args[1])
:except ValueError
raise ValueError("depth must be an integer")
return msg
:"if cmd == "move
:if len(args) != 1
raise ValueError("usage: move <uci>  e.g. move e2e4")
return {"type": "move", "uci": args[0]}
:"if cmd == "resign
return {"type": "resign"}
raise ValueError(f"unknown command: {cmd!r}")
```

src/secure_chess/client/gui.py

```
.Tkinter GUI client for the secure-chess server"""

Three screens managed by a single Tk root: Login → Lobby → Game. The GUI speaks
exactly the same JSON-Lines wire protocol as the CLI client in
secure_chess.client.cli`, so any combination of GUI and CLI clients can play`
.against each other or against the AI

Threading: a single background reader thread drains the TCP socket and pushes
every server message onto a `queue.Queue`. The Tk main loop polls that queue
every 50ms via `root.after`, ensuring every widget mutation happens on the Tk
.main thread
"""
```

```
from __future__ import annotations

import queue
import socket
import threading
import tkinter as tk
from tkinter import messagebox, ttk
from typing import Any, Dict, List, Optional, Tuple

from secure_chess.common import protocol
from secure_chess.common.errors import ConnectionClosed, ProtocolError


SQUARE_SIZE = 64
BOARD_MARGIN = 18
BOARD_PIXELS = SQUARE_SIZE * 8

"LIGHT_COLOR = "#EEEED2
"DARK_COLOR = "#769656
"SELECTED_COLOR = "#F7EC74
"LAST_MOVE_COLOR = "#F0D75C

} = PIECE_TO_UNICODE
,"K": "\u2654", "Q": "\u2655", "R": "\u2656", "B": "\u2657", "N": "\u2658", "P": "\u2659"
,"k": "\u265A", "q": "\u265B", "r": "\u265C", "b": "\u265D", "n": "\u265E", "p": "\u265F"
{


:def parse_fen_placement(fen_short: str) -> List[List[Optional[str]]]
.Parse the placement field of a FEN-short string into an 8x8 grid"""

:Returns a 2D list `grid[file][rank_from_top]` where
file 0 = a-file, file 7 = h-file -
rank_from_top 0 = rank 8, rank_from_top 7 = rank 1 -
each cell is either `None` or a one-character FEN piece code -
(uppercase = white, lowercase = black)
"""
"" placement = fen_short.split()[0] if fen_short else
grid: List[List[Optional[str]]] = [[None] * 8 for _ in range(8)]
:for rank_idx, row in enumerate(placement.split("/"))
:if rank_idx >= 8
break
f = 0
:for ch in row
:()if ch.isdigit
f += int(ch)
:elif f < 8
grid[f][rank_idx] = ch
f += 1
return grid
```

```
:def square_to_grid(sq: str) -> Optional[Tuple[int, int]]
""".`Convert algebraic notation (e.g. 'e2') to grid coords `(file, rank_from_top)"""
:if not isinstance(sq, str) or len(sq) != 2
return None
f = ord(sq[0]) - ord("a")
:if not (0 <= f < 8)
return None
:try
r = int(sq[1])
:except ValueError
return None
:if not (1 <= r <= 8)
return None
return f, 8 - r


:def grid_to_square(file: int, rank_from_top: int) -> str
""".Convert grid coords to algebraic notation (e.g. (4,6) -> 'e2')"""
"return f"{chr(ord('a') + file)}{8 - rank_from_top}


:class ServerConnection
.Background TCP reader publishing every message onto a thread-safe queue"""

The queue is consumed by `ChessGui._poll_events` on the Tk main thread, so
.no Tk widget is ever touched by the reader thread
"""

:def __init__(self, host: str, port: int, timeout: float = 5.0) -> None
self._sock = socket.create_connection((host, port), timeout=timeout)
self._sock.settimeout(None)
self._reader = protocol.make_reader(self._sock)
()self._send_lock = threading.Lock
()self.events: "queue.Queue[Dict[str, Any]]" = queue.Queue
()self._stopping = threading.Event
)self._reader_thread = threading.Thread
,target=self._read_loop
,"name="gui-reader
,daemon=True
(
()self._reader_thread.start

:def send(self, msg: Dict[str, Any]) -> None
:with self._send_lock
protocol.send_message(self._sock, msg)

:def _read_loop(self) -> None
:try
:()while not self._stopping.is_set
msg = protocol.recv_message(self._reader)
self.events.put(msg)
:except ConnectionClosed
self.events.put({"type": "_disconnected", "reason": "server closed the connection"})
```

```
:except ProtocolError as exc
self.events.put({"type": "_disconnected", "reason": f"protocol error: {exc}"})

:def close(self) -> None
()self._stopping.set
:try
self._sock.shutdown(socket.SHUT_RDWR)
:except OSError
pass
:try
()self._sock.close
:except OSError
pass


:class ChessGui
""".Top-level Tk application managing the three screens and the server connection"""

:def __init__(self, host: str, port: int) -> None
self.host = host
self.port = port
self.conn: Optional[ServerConnection] = None

self.username: Optional[str] = None
self.pending_auth_username: Optional[str] = None
self.pending_action: Optional[str] = None
self.waiting_in_lobby: bool = False

self.you_are: Optional[str] = None
self.opponent: Optional[str] = None
self.board_grid: List[List[Optional[str]]] = [[None] * 8 for _ in range(8)]
"self.to_move: str = "white
self.in_check: bool = False
self.last_move: Optional[str] = None
self.selected_square: Optional[str] = None
[] = self.move_log: List[str]
self.game_active: bool = False

()self.root = tk.Tk
self.root.title("Secure Chess")
self.root.geometry("780x680")
self.root.minsize(780, 680)

self.container = tk.Frame(self.root)
self.container.pack(fill="both", expand=True)

self.login_frame = LoginFrame(self.container, self)
self.lobby_frame = LobbyFrame(self.container, self)
self.game_frame = GameFrame(self.container, self)

self.root.protocol("WM_DELETE_WINDOW", self.on_close)
()self.show_login
```

```
:def _hide_all(self) -> None
:for f in (self.login_frame, self.lobby_frame, self.game_frame)
()f.pack_forget

:def show_login(self) -> None
()self._hide_all
self.login_frame.pack(fill="both", expand=True)
()self.login_frame.focus_username

:def show_lobby(self) -> None
()self._hide_all
self.lobby_frame.refresh(self.username)
self.lobby_frame.pack(fill="both", expand=True)

:def show_game(self) -> None
()self._hide_all
()self.game_frame.refresh_all
self.game_frame.pack(fill="both", expand=True)

:def ensure_connected(self) -> bool
:if self.conn is not None
return True
:try
self.conn = ServerConnection(self.host, self.port)
:except OSError as exc
)messagebox.showerror
,"Connection failed"
,"f"Cannot connect to {self.host}:{self.port}\n{exc}
(
return False
self.root.after(50, self._poll_events)
return True

:def _poll_events(self) -> None
:if self.conn is None
return
:try
:while True
()msg = self.conn.events.get_nowait
self._handle_message(msg)
:except queue.Empty
pass
:if self.conn is not None
self.root.after(50, self._poll_events)

:def _handle_message(self, msg: Dict[str, Any]) -> None
t = msg.get("type")
:"if t == "_disconnected
self._on_disconnected(msg.get("reason", "connection closed"))
return
:"if t == "ok
self._on_ok(msg)
:"elif t == "error
```

```
self._on_error(msg)
:"elif t == "game_started
self._on_game_started(msg)
:"elif t == "game_state
self._on_game_state(msg)
:"elif t == "game_ended
self._on_game_ended(msg)

:def _on_ok(self, _msg: Dict[str, Any]) -> None
action = self.pending_action
self.pending_action = None
:if self.pending_auth_username is not None
self.username = self.pending_auth_username
self.pending_auth_username = None
()self.show_lobby
return
:"if action == "play_human
self.waiting_in_lobby = True
()self.lobby_frame.refresh_waiting
:"elif action == "cancel_lobby
self.waiting_in_lobby = False
()self.lobby_frame.refresh_waiting

:def _on_error(self, msg: Dict[str, Any]) -> None
code = msg.get("code", "?")
message = msg.get("message", "?")
action = self.pending_action
self.pending_action = None
:if self.pending_auth_username is not None
self.pending_auth_username = None
messagebox.showerror(f"Login error: {code}", message)
return
messagebox.showerror(f"Error: {code}", message)
:"if action == "cancel_lobby
()self.lobby_frame.refresh_waiting
return
:if self.game_active
self.selected_square = None
()self.game_frame.refresh_board

:def _on_game_started(self, msg: Dict[str, Any]) -> None
self.you_are = msg.get("you_are")
self.opponent = msg.get("opponent")
self.board_grid = parse_fen_placement(msg.get("board", ""))
self.to_move = msg.get("to_move", "white")
self.in_check = False
self.last_move = None
self.selected_square = None
[] = self.move_log
self.game_active = True
self.waiting_in_lobby = False
self.pending_action = None
()self.show_game
```

```
:def _on_game_state(self, msg: Dict[str, Any]) -> None
board = msg.get("board")
:if board
self.board_grid = parse_fen_placement(board)
self.to_move = msg.get("to_move", self.to_move)
self.in_check = bool(msg.get("in_check", False))
last = msg.get("last_move")
:if last
self.last_move = last
self.move_log.append(last)
self.selected_square = None
()self.game_frame.refresh_all

:def _on_game_ended(self, msg: Dict[str, Any]) -> None
result = msg.get("result", "?")
reason = msg.get("reason", "?")
self.game_active = False
messagebox.showinfo("Game ended", f"Result: {result}\nReason: {reason}")
()self.show_lobby

:def _on_disconnected(self, reason: str) -> None
self.conn = None
was_authed = self.username is not None
()self._reset_state
:if was_authed
messagebox.showwarning("Disconnected", reason)
()self.show_login

:def _reset_state(self) -> None
:if self.conn is not None
()self.conn.close
self.conn = None
self.username = None
self.pending_auth_username = None
self.pending_action = None
self.waiting_in_lobby = False
self.you_are = None
self.opponent = None
self.selected_square = None
self.game_active = False
[] = self.move_log
self.board_grid = [[None] * 8 for _ in range(8)]

:def send_command(self, msg: Dict[str, Any]) -> bool
:if self.conn is None
return False
action_type = msg.get("type")
:if action_type in ("play_human", "play_ai", "cancel_lobby")
self.pending_action = action_type
:try
self.conn.send(msg)
return True
```

```
:except (OSError, ConnectionClosed) as exc
self.pending_action = None
messagebox.showerror("Send failed", str(exc))
self._on_disconnected(str(exc))
return False

:def on_close(self) -> None
:if self.conn is not None
:try
self.conn.send({"type": "quit"})
:except (OSError, ConnectionClosed)
pass
()self.conn.close
self.conn = None
()self.root.destroy

:def run(self) -> int
()self.root.mainloop
return 0


:class LoginFrame(tk.Frame)
""".First screen: connect + authenticate (register or login)"""

:def __init__(self, parent: tk.Widget, app: ChessGui) -> None
super().__init__(parent)
self.app = app

title = tk.Label(self, text="Secure Chess", font=("Arial", 32, "bold"))
title.pack(pady=(90, 6))

)subtitle = tk.Label
,self
,"text="Sign in or create an account to start playing
,font=("Arial", 12)
,"fg="#444
(
subtitle.pack(pady=(0, 30))

form = tk.Frame(self)
()form.pack

)tk.Label(form, text="Username:", font=("Arial", 11)).grid
row=0, column=0, sticky="e", padx=5, pady=6
(
self.user_entry = tk.Entry(form, font=("Arial", 11), width=26)
self.user_entry.grid(row=0, column=1, padx=5, pady=6)

)tk.Label(form, text="Password:", font=("Arial", 11)).grid
row=1, column=0, sticky="e", padx=5, pady=6
(
self.pwd_entry = tk.Entry(form, font=("Arial", 11), width=26, show="\u2022")
self.pwd_entry.grid(row=1, column=1, padx=5, pady=6)
```

```
self.pwd_entry.bind("<Return>", lambda _e: self._login())

btn_row = tk.Frame(self)
btn_row.pack(pady=22)

)tk.Button
btn_row, text="Login", font=("Arial", 11), width=14, command=self._login
pack(side="left", padx=6).(
)tk.Button
btn_row, text="Register", font=("Arial", 11), width=14, command=self._register
pack(side="left", padx=6).(

)self.hint = tk.Label
,self
,"text=f"Server: {app.host}:{app.port}
,font=("Arial", 9)
,"fg="#888
(
self.hint.pack(side="bottom", pady=8)

:def focus_username(self) -> None
()self.user_entry.focus_set

:def _validate(self) -> Optional[Tuple[str, str]]
()u = self.user_entry.get().strip
()p = self.pwd_entry.get
:if not u or not p
messagebox.showerror("Missing fields", "Both username and password are required.")
return None
return u, p

:def _login(self) -> None
()vals = self._validate
:if vals is None
return
:()if not self.app.ensure_connected
return
u, p = vals
self.app.pending_auth_username = u
self.app.send_command({"type": "login", "username": u, "password": p})

:def _register(self) -> None
()vals = self._validate
:if vals is None
return
:()if not self.app.ensure_connected
return
u, p = vals
self.app.pending_auth_username = u
self.app.send_command({"type": "register", "username": u, "password": p})


:class LobbyFrame(tk.Frame)
```

```
.Second screen: choose to play another human or play against the AI"""

:`Tracks two visual sub-states driven by `ChessGui.waiting_in_lobby
.normal: both Join Lobby and Start AI Game are enabled -
waiting: a yellow banner with "Waiting for opponent..." is shown and the -
.two action buttons are greyed out; only Cancel + Logout remain clickable
"""

:def __init__(self, parent: tk.Widget, app: ChessGui) -> None
super().__init__(parent)
self.app = app

self.title_label = tk.Label(self, text="", font=("Arial", 22, "bold"))
self.title_label.pack(pady=(40, 6))

)tk.Label
,self
,"text="Choose your opponent
,font=("Arial", 12)
,"fg="#444
pack(pady=(0, 12)).(

self.waiting_banner = tk.Frame(self, bg="#FFF4C2", bd=1, relief="solid")
)self.waiting_label = tk.Label
,self.waiting_banner
,"...text="\u23F3  Waiting for another player to join the lobby
,font=("Arial", 11, "bold")
,"bg="#FFF4C2
,"fg="#664D03
(
self.waiting_label.pack(side="left", padx=12, pady=8)
)self.cancel_btn = tk.Button
,self.waiting_banner
,"text="Cancel
,font=("Arial", 10)
,command=self._cancel_lobby
(
self.cancel_btn.pack(side="right", padx=10, pady=6)

)human_frame = tk.LabelFrame
self, text=" Play Human ", font=("Arial", 11, "bold"), padx=18, pady=14
(
human_frame.pack(pady=10, padx=80, fill="x")
)tk.Label
,human_frame
)=text
".Wait in the lobby until another authenticated player joins"
".nColors are assigned automatically (first to wait plays White)\"
,(
,"justify="left
pack(anchor="w", pady=(0, 8)).(
)self.join_btn = tk.Button
,human_frame
```

```
,"text="Join Lobby
,font=("Arial", 11)
,width=18
,command=self._play_human
(
()self.join_btn.pack

)ai_frame = tk.LabelFrame
self, text=" Play AI (bonus) ", font=("Arial", 11, "bold"), padx=18, pady=14
(
ai_frame.pack(pady=10, padx=80, fill="x")

opts = tk.Frame(ai_frame)
opts.pack(anchor="w", pady=(0, 8))

tk.Label(opts, text="Your color:").grid(row=0, column=0, sticky="e", padx=4, pady=2)
self.color_var = tk.StringVar(value="white")
)tk.Radiobutton(opts, text="White", variable=self.color_var, value="white").grid
"row=0, column=1, sticky="w
(
)tk.Radiobutton(opts, text="Black", variable=self.color_var, value="black").grid
"row=0, column=2, sticky="w
(

tk.Label(opts, text="AI search depth:").grid(row=1, column=0, sticky="e", padx=4, pady=2)
self.depth_var = tk.IntVar(value=3)
)ttk.Combobox
,opts
,textvariable=self.depth_var
,values=[1, 2, 3, 4]
,width=4
,"state="readonly
grid(row=1, column=1, sticky="w").(
)tk.Label(opts, text="(higher = stronger but slower)", fg="#666").grid
row=1, column=2, columnspan=2, sticky="w", padx=(8, 0)
(

)self.ai_btn = tk.Button
,ai_frame
,"text="Start AI Game
,font=("Arial", 11)
,width=18
,command=self._play_ai
(
()self.ai_btn.pack

)tk.Button(self, text="Logout", font=("Arial", 10), command=self._logout).pack
side="bottom", pady=16
(

:def refresh(self, username: Optional[str]) -> None
self.title_label.config(text=f"Welcome, {username or 'guest'}")
()self.refresh_waiting
```

```
:def refresh_waiting(self) -> None
:if self.app.waiting_in_lobby
self.waiting_banner.pack(after=self.title_label, padx=80, pady=(4, 16), fill="x")
self.join_btn.config(state="disabled")
self.ai_btn.config(state="disabled")
:else
()self.waiting_banner.pack_forget
self.join_btn.config(state="normal")
self.ai_btn.config(state="normal")

:def _play_human(self) -> None
:if self.app.waiting_in_lobby
return
self.app.send_command({"type": "play_human"})

:def _play_ai(self) -> None
:if self.app.waiting_in_lobby
)messagebox.showinfo
,"Already in lobby"
"You're currently waiting for a human opponent.\n"
,".Click Cancel first if you'd rather play against the AI"
(
return
)self.app.send_command
}
,"type": "play_ai"
,()color": self.color_var.get"
,depth": int(self.depth_var.get())"
{
(

:def _cancel_lobby(self) -> None
:if not self.app.waiting_in_lobby
return
self.app.send_command({"type": "cancel_lobby"})

:def _logout(self) -> None
:if self.app.conn is not None
:try
self.app.conn.send({"type": "quit"})
:except (OSError, ConnectionClosed)
pass
()self.app._reset_state
()self.app.show_login


:class GameFrame(tk.Frame)
""".Third screen: status bar, clickable 8x8 board, move log, resign button"""

:def __init__(self, parent: tk.Widget, app: ChessGui) -> None
super().__init__(parent)
self.app = app
```

```
)self.status_label = tk.Label
"self, text="", font=("Arial", 11), anchor="w", justify="left
(
self.status_label.pack(fill="x", padx=12, pady=(10, 4))

body = tk.Frame(self)
body.pack(padx=12, pady=4, fill="both", expand=True)

canvas_width = BOARD_PIXELS + BOARD_MARGIN * 2
canvas_height = BOARD_PIXELS + BOARD_MARGIN * 2
)self.canvas = tk.Canvas
,body
,width=canvas_width
,height=canvas_height
,"bg="#f5f5f0
,highlightthickness=0
(
self.canvas.pack(side="left")
self.canvas.bind("<Button-1>", self._on_click)

side = tk.Frame(body)
side.pack(side="left", padx=(16, 0), fill="y")

tk.Label(side, text="Move log", font=("Arial", 10, "bold")).pack(anchor="w")
log_box = tk.Frame(side)
log_box.pack(pady=(2, 10))
)self.log_text = tk.Text
log_box, width=18, height=22, state="disabled", font=("Courier", 10)
(
self.log_text.pack(side="left")
scroll = tk.Scrollbar(log_box, command=self.log_text.yview)
scroll.pack(side="right", fill="y")
self.log_text.config(yscrollcommand=scroll.set)

tk.Button(side, text="Resign", width=16, command=self._resign).pack(pady=(2, 4))
()tk.Button(side, text="Quit to Lobby", width=16, command=self._lobby).pack

:def refresh_all(self) -> None
()self.refresh_status
()self.refresh_board
()self.refresh_log

:def refresh_status(self) -> None
"?" you = self.app.you_are or
"?" opp = self.app.opponent or
to_move = self.app.to_move
"—" last = self.app.last_move or
your_turn = to_move == self.app.you_are
"" turn_marker = "  ← YOUR TURN" if your_turn else
"" check_text = "    \u26A0 CHECK" if self.app.in_check else
) = text
"f"You: {you}    Opponent: {opp}    To move: {to_move}{turn_marker}{check_text}\n
```

```
"f"Last move: {last}
(
self.status_label.config(text=text, fg="#A00" if self.app.in_check else "#000")

:def _board_orientation_flipped(self) -> bool
"return self.app.you_are == "black

:def _logical_to_display(self, f: int, rt: int) -> Tuple[int, int]
:()if self._board_orientation_flipped
return 7 - f, 7 - rt
return f, rt

:def _display_to_logical(self, df: int, drt: int) -> Tuple[int, int]
:()if self._board_orientation_flipped
return 7 - df, 7 - drt
return df, drt

:def refresh_board(self) -> None
c = self.canvas
c.delete("all")
s = SQUARE_SIZE
m = BOARD_MARGIN

:for f in range(8)
:for rt in range(8)
df, drt = self._logical_to_display(f, rt)
x0 = m + df * s
y0 = m + drt * s
light = (f + rt) % 2 == 0
color = LIGHT_COLOR if light else DARK_COLOR
c.create_rectangle(x0, y0, x0 + s, y0 + s, fill=color, outline="")

:if self.app.last_move and len(self.app.last_move) >= 4
:for sq in (self.app.last_move[:2], self.app.last_move[2:4])
fr = square_to_grid(sq)
:if fr is None
continue
f, rt = fr
df, drt = self._logical_to_display(f, rt)
x0 = m + df * s
y0 = m + drt * s
)c.create_rectangle
,x0 + 2, y0 + 2, x0 + s - 2, y0 + s - 2
,outline=LAST_MOVE_COLOR, width=3
(

:if self.app.selected_square is not None
fr = square_to_grid(self.app.selected_square)
:if fr is not None
f, rt = fr
df, drt = self._logical_to_display(f, rt)
x0 = m + df * s
y0 = m + drt * s
```

```
c.create_rectangle(x0, y0, x0 + s, y0 + s, fill=SELECTED_COLOR, outline="")

:for f in range(8)
:for rt in range(8)
piece = self.app.board_grid[f][rt]
:if piece is None
continue
df, drt = self._logical_to_display(f, rt)
cx = m + df * s + s // 2
cy = m + drt * s + s // 2
glyph = PIECE_TO_UNICODE.get(piece, piece)
:()if piece.isupper
c.create_text(cx + 1, cy + 1, text=glyph, font=("Arial", 40), fill="#000")
c.create_text(cx, cy, text=glyph, font=("Arial", 40), fill="#FFF")
:else
c.create_text(cx, cy, text=glyph, font=("Arial", 40), fill="#000")

:for i in range(8)
df, _ = self._logical_to_display(i, 0)
fx = m + df * s + s // 2
)c.create_text
,fx, m + 8 * s + 9
,text=chr(ord("a") + i)
,"font=("Arial", 10), fill="#333
(
:for i in range(8)
drt = self._logical_to_display(0, i) ,_
ry = m + drt * s + s // 2
c.create_text(m - 10, ry, text=str(8 - i), font=("Arial", 10), fill="#333")

:def refresh_log(self) -> None
self.log_text.config(state="normal")
self.log_text.delete("1.0", tk.END)
:for i, move in enumerate(self.app.move_log)
move_no = i // 2 + 1
:if i % 2 == 0
self.log_text.insert(tk.END, f"{move_no:>3}. {move:<6} ")
:else
self.log_text.insert(tk.END, f"{move}\n")
self.log_text.see(tk.END)
self.log_text.config(state="disabled")

:def _on_click(self, event: tk.Event) -> None
:if not self.app.game_active
return
x = event.x - BOARD_MARGIN
y = event.y - BOARD_MARGIN
:if not (0 <= x < BOARD_PIXELS and 0 <= y < BOARD_PIXELS)
return
df = x // SQUARE_SIZE
drt = y // SQUARE_SIZE
f, rt = self._display_to_logical(df, drt)
sq = grid_to_square(f, rt)
```

```
:if self.app.to_move != self.app.you_are
return

piece_here = self.app.board_grid[f][rt]
own_piece_here = piece_here is not None and self._is_own_piece(piece_here)

:if self.app.selected_square is None
:if own_piece_here
self.app.selected_square = sq
()self.refresh_board
return

:if sq == self.app.selected_square
self.app.selected_square = None
()self.refresh_board
return

:if own_piece_here
self.app.selected_square = sq
()self.refresh_board
return

origin = self.app.selected_square
uci = origin + sq

origin_fr = square_to_grid(origin)
:if origin_fr is not None
of, ort = origin_fr
moving = self.app.board_grid[of][ort]
:if moving == "P" and rt == 0
"uci += "q
:elif moving == "p" and rt == 7
"uci += "q

self.app.send_command({"type": "move", "uci": uci})

:def _is_own_piece(self, piece: str) -> bool
:"if self.app.you_are == "white
()return piece.isupper
:"if self.app.you_are == "black
()return piece.islower
return False

:def _resign(self) -> None
:if not self.app.game_active
return
:if messagebox.askyesno("Resign", "Are you sure you want to resign?")
self.app.send_command({"type": "resign"})

:def _lobby(self) -> None
:if not self.app.game_active
()self.app.show_lobby
```

```
return
:if messagebox.askyesno("Quit Game", "Quitting will count as resigning. Continue?")
self.app.send_command({"type": "resign"})


:def run_gui(host: str, port: int) -> int
""".`__Entry point used by `secure_chess.client.__main"""
()return ChessGui(host, port).run
```

נספח ב' — תוצאות הרצת בדיקות (pytest)

הפלט להלן הופק ע"י הרצת "pytest -v" על המאגר. כל 99 הבדיקות עוברות בכ-12 שניות.

```
============================= test session starts =============================
platform win32 -- Python 3.12.0, pytest-9.0.3, pluggy-1.6.0
rootdir: C:\Users\Alonti\Documents\GitHub\secure-chess
configfile: pytest.ini
testpaths: tests
collected 99 items

tests\integration\test_cancel_lobby.py ...                               [  3%]
tests\integration\test_full_game.py ...                                  [  6%]
tests\integration\test_parallel_games.py ..                              [  8%]
tests\integration\test_play_ai.py .                                      [  9%]
tests\integration\test_register_login.py ...                             [ 12%]
tests\unit\test_ai.py .....                                              [ 23%]
tests\unit\test_board_rules.py .....                                     [ 38%]
tests\unit\test_client_gui.py .....                                      [ 60%]
tests\unit\test_crypto.py ......                                         [ 66%]
tests\unit\test_pieces.py .........                                      [ 75%]
tests\unit\test_protocol_auth.py .......                                 [ 82%]
tests\unit\test_protocol_game.py ........                                [ 90%]
tests\unit\test_user_store.py .........                                  [100%]

============================= passed in 12.20s 99 =============================
```

נספח ג' — טבלת הודעות הפרוטוקול המלאה

ג.1 הודעות לקוח → שרת

סוג הודעה	תיאור	שדות חובה
register	הרשמת משתמש חדש	username, password
login	התחברות לחשבון קיים	username, password
play_human	הצטרפות ל-lobby להמתנה לשחקן	(אין)
play_ai	התחלת משחק נגד AI	color, depth (אופציונלי)
cancel_lobby	ביטול ההמתנה ב-lobby	(אין)
move	ביצוע מהלך במשחק פעיל	uci (e.g. 'e2e4', 'e7e8q')
resign	התפטרות ממשחק פעיל	(אין)
quit	סגירת חיבור מסודרת	(אין)

ג.2 הודעות שרת → לקוח

סוג הודעה	תיאור	שדות
ok	אישור הצלחה לפעולה האחרונה	(אופציונלי תוכן נוסף)
error	שגיאה במהלך עיבוד הפעולה	code, message
game_started	המשחק התחיל	game_id, you_are, opponent, board (FEN), to_move
game_state	עדכון מצב המשחק (אחרי מהלך)	game_id, last_move, board, to_move, in_check
game_ended	סיום המשחק	game_id, result, reason

ג.3 דוגמת זרימה מלאה

```
C -> S  {"type":"register","username":"alice","password":"hunter2!"}
S -> C  {"type":"ok"}

C -> S  {"type":"play_human"}
S -> C  {"type":"ok"}

:lobby-גם הצטרף ל bob-# אחרי ש
,"S -> C  {"type":"game_started","game_id":"abc123…","you_are":"white
,"opponent":"bob","board":"rnbqkbnr/pppppppp/8/8/8/8/PPPPPPPP/RNBQKBNR"
{"to_move":"white"

C -> S  {"type":"move","uci":"e2e4"}
```

```
S -> C  {"type":"ok"}
,"S -> C  {"type":"game_state","last_move":"e2e4
,"board":"rnbqkbnr/pppppppp/8/8/4P3/8/PPPP1PPP/RNBQKBNR"
{to_move":"black","in_check":false"

:סופית ... מהלכים נוספים ... #
S -> C  {"type":"game_ended","result":"white_wins","reason":"checkmate"}
```

נספח ד' — שאלות תיאורטיות אפשריות לבחינה

להלן רשימה של שאלות תיאורטיות אפשריות מהבוחן עם תשובות מוכנות, מסווגות לפי קטגוריה.

ד.1 תקשורת

ש: איזו שכבות קיימות במודל התקשורת?

ת: מודל OSI: 7 — Application (HTTP, JSON-Lines שלי), 6 — Presentation (UTF-8 JSON), 5 — Session (TCP), 4 — Transport (TCP), 3 — Network (IP), 2 — Data Link (Ethernet), 1 — Physical.

ש: באיזה פרוטוקול תקשורת אתה עובד ואיך קבעת זאת?

ת: TCP, נקבע ביצירת הסוקט עם socket.socket(socket.AF_INET, socket.SOCK_STREAM). TCP בגלל שאני צריך אמינות וסדר במהלכי שחמט.

ש: מה ההבדל בין TCP ל-UDP?

ת: TCP: אמין, מסודר, connection-oriented, אטי יחסית (handshake), reliable retransmission. UDP: לא אמין, לא מסודר, ללא חיבור, מהיר, מתאים ל-streaming.

ש: מהי לחיצת יד משולשת?

ת: Three-way handshake של TCP: SYN → SYN+ACK → ACK. שלוש הודעות שמסכימים על סדרות התחלה לפני העברת נתונים.

ש: מהי כתובת IP ולמה משמש PORT?

ת: כתובת IP מזהה מחשב ברשת (לדוגמה 127.0.0.1). PORT מזהה תהליך/שירות באותו מחשב (לדוגמה 5050 = השרת שלנו). יחד הם מהווים endpoint יחיד.

ש: דוגמאות לפרוטוקולים בשכבת האפליקציה?

ת: HTTP (web), SMTP (email), DNS (domain name), FTP (file transfer), SSH (remote shell), ובפרויקט שלי — JSON-Lines משלי.

ש: למה משמשת הפקודה PING?

ת: שולחת ICMP Echo Request למחשב מטרה ומחכה ל-Echo Reply. שימושית לבדוק קישוריות ולמדוד זמן round-trip.

ד.2 קריפטוגרפיה

ש: איזה סוגי הצפנות קיימים?

ת: Symmetric (אותו מפתח להצפנה+פענוח, מהיר — AES, ChaCha20) ו-Asymmetric (זוג מפתחות ציבורי+פרטי, איטי — RSA, ECC). בנוסף, hashing הוא חד-כיווני (לא הצפנה אלא טביעת אצבע — SHA-256, bcrypt).

ש: מה זה מפתח ציבורי ומה זה מפתח פרטי?

ת: במערכת asymmetric יש זוג מפתחות מתמטית קשורים. כל מה שמוצפן בציבורי אפשר לפענח רק בפרטי, ולהיפך. הציבורי ניתן לכולם; הפרטי נשמר בסוד.

ש: מה זה חתימה דיגיטלית?

ת: מחשבים hash של ההודעה ומצפינים אותו במפתח הפרטי של החותם. כל אחד עם המפתח הציבורי יכול לאמת שזה באמת אותו אדם חתם (אי-הכחשה).

ש: מה זה hash?

ת: פונקציה חד-כיוונית שמקבלת קלט בכל גודל ומחזירה פלט בגודל קבוע (לדוגמה 256 ביט). תכונות: deterministic, fast to compute, hard to invert, collision-resistant. שימושים: צ'ק-סאם, סיסמאות (bcrypt), חתימות, blockchain.

ש: למה bcrypt עדיף על SHA-256 לסיסמאות?

ת: bcrypt (1) אטי בכוונה (work factor), כך שתוקף לא יכול לבדוק מיליארדי ניחושים בשנייה. (2) bcrypt מוסיף salt אקראי לכל סיסמה, כך שאפילו סיסמאות זהות מקבלות hashes שונים — rainbow tables חסרי תועלת.

ש: האם הסיסמה עוברת מוצפנת ברשת בפרויקט שלך?

ת: לא — הסיסמה עוברת ב-JSON-Lines בקליר על TCP. לפי הנחיית המורה הראשונית, הפרויקט נדרש להצפנת סיסמאות במנוחה בלבד (bcrypt על הדיסק) ולא הצפנת תקשורת. להגנה אמיתית בפרודקשן הייתי עוטף את ה-TCP ב-TLS.

ד.3 מערכות הפעלה

ש: מה זה thread?

ת: יחידת ביצוע בתוך תהליך (process). threads באותו process חולקים זיכרון (global vars, heap) אבל יש להם stack משלהם. ה-OS מתזמן threads למעבד באמצעות preemptive scheduling.

ש: מה זה process?

ת: מופע פעיל של תוכנית. לכל process יש זיכרון משלו, PID, טבלת קבצים פתוחים, ומשתני סביבה. ל-OS עולה הרבה יותר לעבור בין processes (context switch) מאשר בין threads.

ש: מה זה WinAPI?

ת: הממשק התכנותי של Windows — כ-9000 פונקציות לגישה לכל יכולת של המערכת (קבצים, תהליכים, חלונות, רשת, …). Tkinter ב-Python קורא תחתית WinAPI דרך פקדי Win32 בפועל.

ש: מה זה GIL ולמה הוא חשוב?

ת: Global Interpreter Lock — לוק יחיד ב-CPython שמבטיח שרק thread אחד מריץ byte-code בכל רגע. תוצאה: threads ב-Python לא מקבילים על CPU, אבל בזמן I/O (socket.recv, time.sleep) ה-GIL משוחרר — אז threading עדיין שימושי ל-I/O-bound code כמו השרת שלנו.

ד.4 קבצים

ש: מה זה magic number בקובץ?

ת: מספר/חתימה קצרה בהתחלת קובץ שמזהה את סוגו. דוגמאות: PNG (0x89 50 4E 47), PDF (%PDF-), ZIP (PK\x03\x04), PE/EXE (MZ). מפעיל הקובץ קורא את ה-magic לפני שהוא מנסה לעבד אותו.

ש: מה זה FAT32?

ת: מערכת קבצים ישנה (Microsoft, 1996) המבוססת על File Allocation Table. מגבלות: קובץ בודד ≤ 4GB, partition ≤ 8TB, אין הרשאות נפרדות. עדיין נפוץ ב-USB drives וב-SD cards בגלל תאימות רחבה.

ש: מה זה קובץ PE (קובץ הרצה)?

ת: Portable Executable — פורמט הקבצים הניתנים להרצה ב-Windows (.exe, .dll, .sys). מבנה: DOS stub → PE header → section table → sections (.text, .data, .rsrc). הטוען של Windows קורא את ה-PE header ומעמיס את הקובץ לזיכרון לפני שהוא מעביר שליטה ל-entry point.

ד.5 סייבר — אבטחת הפרויקט

ש: לאלו מתקפות הפרויקט חשוף?

ת: (1) האזנה לתעבורה — סיסמה ומהלכים בקליר. (2) MITM — תוקף יכול לזייף מהלכים. (3) Brute-force מקוון על login — bcrypt מאט אבל לא חוסם לחלוטין. (4) DoS אפליקטיבי — לקוח יכול לפתוח חיבורים רבים. אלו חולשות שמודעים אליהן ומחוץ ל-scope של הפרויקט לפי הנחיית המורה.

ש: אלו הגנות יושמו בפרויקט?

ת: (1) bcrypt לסיסמאות במנוחה. (2) Validation של כל שדה מהלקוח. (3) State machine — לא ניתן לשלוח move בלי להיות IN_GAME. (4) try/except — נפילת לקוח לא מפילה את השרת. (5) threading.Lock על כל מבנה משותף. (6) atomic write ל-users.json עם os.replace + os.fsync. (7) AlreadyLoggedInError — חיבור כפול לאותו חשבון נחסם.

ש: מה זה MITM?

ת: Man-In-The-Middle — תוקף שיושב בין הלקוח לשרת, מאזין ואולי משנה הודעות. הגנה: TLS עם תעודות שמאמתות את השרת. בפרויקט שלי TLS לא מומש (לפי הנחיית המורה), אז יש להריץ רק על localhost / VPN פנימי.

ש: מה זה SQL Injection?

ת: התקפה שבה תוקף מזריק קוד SQL לתוך שדה קלט שמשולב ב-query. תוצאה: יכול לקרוא, לשנות או למחוק את כל ה-DB. הגנה: prepared statements. בפרויקט שלי אין SQL כלל — אחסון = JSON עם json.dumps שמקודד תווים מיוחדים.

ש: מה זה Buffer Overflow?

ת: כתיבה מעבר לסוף buffer בזיכרון, שעלולה לדרוס return address ולגרום להרצת קוד שרירותי. רלוונטי לשפות עם pointer arithmetic (C/C++). בפרויקט שלי (Python) לא רלוונטי — אין pointer arithmetic, רק bounded slices.

ש: איך נשמרת אבטחת הנתונים בפרויקט?

ת: (1) סיסמאות — hashed עם bcrypt + salt בקובץ Server-authoritative data/users.json. (2) כל מהלך מאומת בצד שרת לפני שמופעל על הלוח. (3) State machine: לקוח לא יכול לקפוץ בין מצבים (BadStateError). (4) atomic write — קובץ users.json לעולם לא נשאר חצי-כתוב גם בקריסה.